SMART FUEL DISPATCH & FLEET TELEMETRY SYSTEM

23CS54C - MODERN WEB TECHNOLOGIES

Submitted by

PRAVIN N 24104080

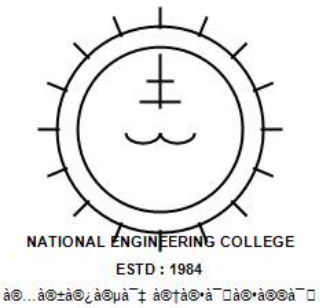
In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING/TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING/ INFORMATION TECHNOLOGY



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

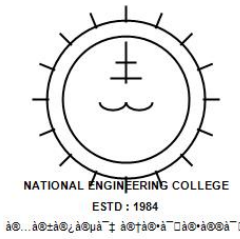
K.R.NAGAR, KOVILPATTI - 628503

OCTOBER - 2026

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503



BONAFIDE CERTIFICATE

This is to certify that this project report, "Smart Fuel Dispatch & Fleet Telemetry System", is the bonafide work of PRAVIN N (24104080) who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the 23CS54C - MODERN WEB TECHNOLOGIES Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

CH NO TITLE..... PAGE NO

1 INTRODUCTION.....3

2 OBJECTIVES..... 4

3 DESCRIPTION.....5

4 CLASS DIAGRAM.....6

5 TABLE STRUCTURES..... 7

6 ER DIAGRAM..... 16

7 TECH STACK.....18

8 MODULES..... 19

9 CONCLUSION.....70

CHAPTER 1
INTRODUCTION

In the modern industrial energy supply chain, the efficient management of bulk fuel distribution, terminal storage operations, and fleet logistics plays a critical role in sustaining manufacturing industries, heavy transport networks, hospitals, and commercial enterprises. Historically, bulk fuel distribution operations have relied on manual paper-based logbooks, telephone-driven dispatch instructions, disjointed spreadsheets, and uncalibrated transport manifests. Such conventional practices are highly prone to human error, introduce substantial communication latency, obscure inventory traceability, and elevate operational risks such as stockouts, consignment misrouting, unauthorised cargo diversion, and regulatory non-compliance with the Petroleum and Explosives Safety Organisation (PESO) standards. There is consequently an urgent imperative for an enterprise-grade digital solution that consolidates depot operations, order processing, and fleet tracking under a unified, real-time platform.

The Smart Fuel Dispatch & Fleet Telemetry System (FDMS) is a comprehensive, full-stack enterprise web application designed to digitize, orchestrate, and automate the end-to-end lifecycle of commercial fuel delivery operations. Spanning from corporate customer purchase order placement and statutory KYC verification to intelligent heuristic tanker allocation, real-time GIS vehicle telemetry, and automated GST tax invoicing, the platform unifies terminal management into a synchronized operational ecosystem. The system is engineered to satisfy the real-world logistics and statutory requirements of commercial petroleum distributors across Tamil Nadu, operating continuously in a high-availability environment.

The application architecture is structured into three foundational functional pillars: Terminal Governance & Master Data Management, which maintains centralized records for regional depots, dedicated tanker fleets, certified hazardous cargo drivers, wholesale fuel tariff schedules, and corporate customer KYC profiles; Operational Logistics & Smart Dispatch Engine, which orchestrates the complete order lifecycle from pending customer requisition through algorithmic tanker assignment, terminal vault fuel reservation, delivery queue scheduling, and digital proof-of-discharge capture; and Live Telemetry, Driver Portal & Business Analytics, which provides real-time GPS coordinate broadcasts over WebSockets (Socket.IO), interactive Leaflet GIS radar tracking, automated low-stock vault threshold alerts, and dynamic exportable management reports.

The system is built upon a high-performance technology stack comprising React 18 with Vite and TailwindCSS on the frontend, a modular Node.js and Express.js v5 RESTful API engine on the backend, MongoDB Atlas with 2dsphere geospatial indexing for persistence, and bidirectional Socket.IO WebSockets for low-latency vehicle telemetry streaming. The infrastructure is supported by an automated Jenkins CI/CD pipeline defined in a declarative Jenkinsfile, ensuring reproducible automated testing, build artifact generation, and zero-downtime containerized production deployments. This report documents the design, architecture, database schemas, and implementation of the Smart Fuel Dispatch & Fleet Telemetry System, providing a comprehensive technical evaluation of its features and capabilities.

CHAPTER 2

OBJECTIVES

- To Digitize Bulk Fuel Supply Chain & Dispatch Operations: Replace manual paper manifests, phone calls, and error-prone logbooks with a centralized web platform for seamless order handling and operational efficiency.
- To Centralize Multi-Depot Master Registers: Maintain synchronized and validated master datasets across 5 regional fuel hubs (Chennai, Salem, Madurai, Trichy, Coimbatore), calibrated tankers, licensed drivers, and official wholesale tariffs.
- To Enforce Granular Role-Based Access Control (RBAC): Secure terminal operations with JSON Web Token (JWT) authentication across 6 distinct user roles (Platform Admin, Depot Manager, Support Executive, Auditor, Customer, Driver) with automated depot jurisdiction scoping (scopeToDepot).
- To Automate Commercial Bulk Order Placement: Provide self-service ordering for verified corporate buyers with dynamic wholesale fuel pricing, automated 18% GST calculation, fixed delivery surcharges, and real-time credit limit checking.
- To Engineer an Intelligent Heuristic Dispatch Engine: Automate delivery assignment by evaluating fuel cargo compatibility (DSL, PTL, LPG, KRS), tanker calibrated payload capacity, on-duty driver availability, and terminal geographic proximity.
- To Provide Real-Time Fleet Telemetry & GPS Tracking: Broadcast live vehicle coordinates, speed, and heading over WebSockets (Socket.IO) to an interactive Leaflet GIS map with route history breadcrumbs and dynamic ETA calculations.
- To Monitor Storage Vault Levels & Sensor Telemetry: Provide continuous monitoring of depot storage tanks with real-time level, temperature, and pressure sensor readings, automated low-stock warnings, and structured OMC refill logging.
- To Standardize Corporate KYC Onboarding & Credit Governance: Facilitate digital customer registration with automated PAN/GST verification, uploaded document management, credit limit allocation, and formal administrative approval workflows.
- To Enforce an Immutable Security Audit Trail: Record comprehensive, tamper-proof activity logs for all critical system transactions, rate revisions, user logins, dispatch assignments, and delivery completions with user attribution, role, and IP address tracking.
- To Deploy a Scalable, Maintainable Cloud Architecture via CI/CD: Construct a decoupled, modular client-server architecture with automated builds, testing, and continuous deployment powered by a declarative Jenkins pipeline (Jenkinsfile).

CHAPTER 3
DESCRIPTION

The Smart Fuel Dispatch & Fleet Telemetry System is a modern, full-stack enterprise web application developed to orchestrate, optimize, and monitor commercial petroleum logistics across a network of regional terminal depots. The system is architected across five core functional operational areas:

1. Terminal Governance & Master Data Management

This module maintains centralized, authoritative records across all operational entities. It manages the registry of the 5 regional hubs across Tamil Nadu (Chennai Central, Salem, Madurai, Trichy, Coimbatore), official wholesale fuel price schedules (Diesel, Petrol, LPG, Kerosene), calibrated delivery tanker specifications (cargo type, capacity, tire pressure, temperature limits), certified hazardous transport drivers with on-duty toggles, and corporate customer profiles. Strict master data integrity guarantees consistent transaction validation, pricing formulas, and jurisdictional access control across all depots.

2. Transaction Management & Smart Dispatch Engine

This module orchestrates daily terminal operations throughout the entire consignment lifecycle. Customers submit purchase orders with preferred delivery slots; depot managers review and authorize orders against credit limits and available storage; the heuristic dispatch engine recommends optimal vehicle-driver pairings based on fuel compartment compatibility, vehicle status, and geographic proximity. The module maintains the active dispatch queue, captures driver discharge telemetry and customer digital signatures upon delivery, and automatically generates computerized GST-compliant tax invoices.

3. Storage Vault Telemetry & Inventory Management

This module monitors stationary terminal storage tanks in real time. It records telemetry data including current fuel volumes, tank fill percentages, temperature readings, and pressure levels. The system automatically issues visual warnings and critical alerts when fuel levels breach statutory reorder thresholds (minimum operating stock). Furthermore, depot managers can initiate replenishment requests to Oil Marketing Companies (IOCL, BPCL, HPCL) and maintain a comprehensive historical audit of all bulk vault refills.

4. Security, Access Control & Compliance

Security is enforced at multiple layers using bcryptjs cryptographic hashing for credential protection, stateless JSON Web Tokens (JWT) for session management, and granular Role-Based Access Control (RBAC). The system defines 6 specialized roles: Platform Admin, Depot Manager, Support Executive, Auditor, Customer, and Driver. Middleware-driven terminal scoping (scopeToDepot) ensures depot managers only access records within their assigned terminal jurisdiction. Additionally, an immutable activity log captures an auditable record of all logins, rate changes, approvals, and dispatch actions with IP addresses and user attribution.

5. Technology, WebSockets & Production Deployment

The application utilizes React 18 with Vite and TailwindCSS for a responsive, accessible single-page interface, Node.js with Express.js v5 for a RESTful backend API service, and MongoDB Atlas with Mongoose ODM for scalable document persistence. Real-time telemetry is streamed via bidirectional

Socket.IO WebSockets to update live Leaflet GIS maps. The entire codebase is backed by an automated Jenkins CI/CD pipeline (Jenkinsfile) executing automated builds, lint checks, test suites, and containerized deployment, ensuring robust continuous integration and high availability.

CHAPTER 4
CLASS DIAGRAM

The Class Diagram models the static structural design of the Smart Fuel Dispatch & Fleet Telemetry System. It represents the core system entities, domain models, attributes, method signatures, and associations across the operational lifecycle. The diagram illustrates key relationships such as the composition of Orders from Customers and Depots, association of Orders with assigned Vehicles and Drivers, association of Vehicles with continuous DriverLocation breadcrumbs, and link between Inventory Tanks and historical Refill records.

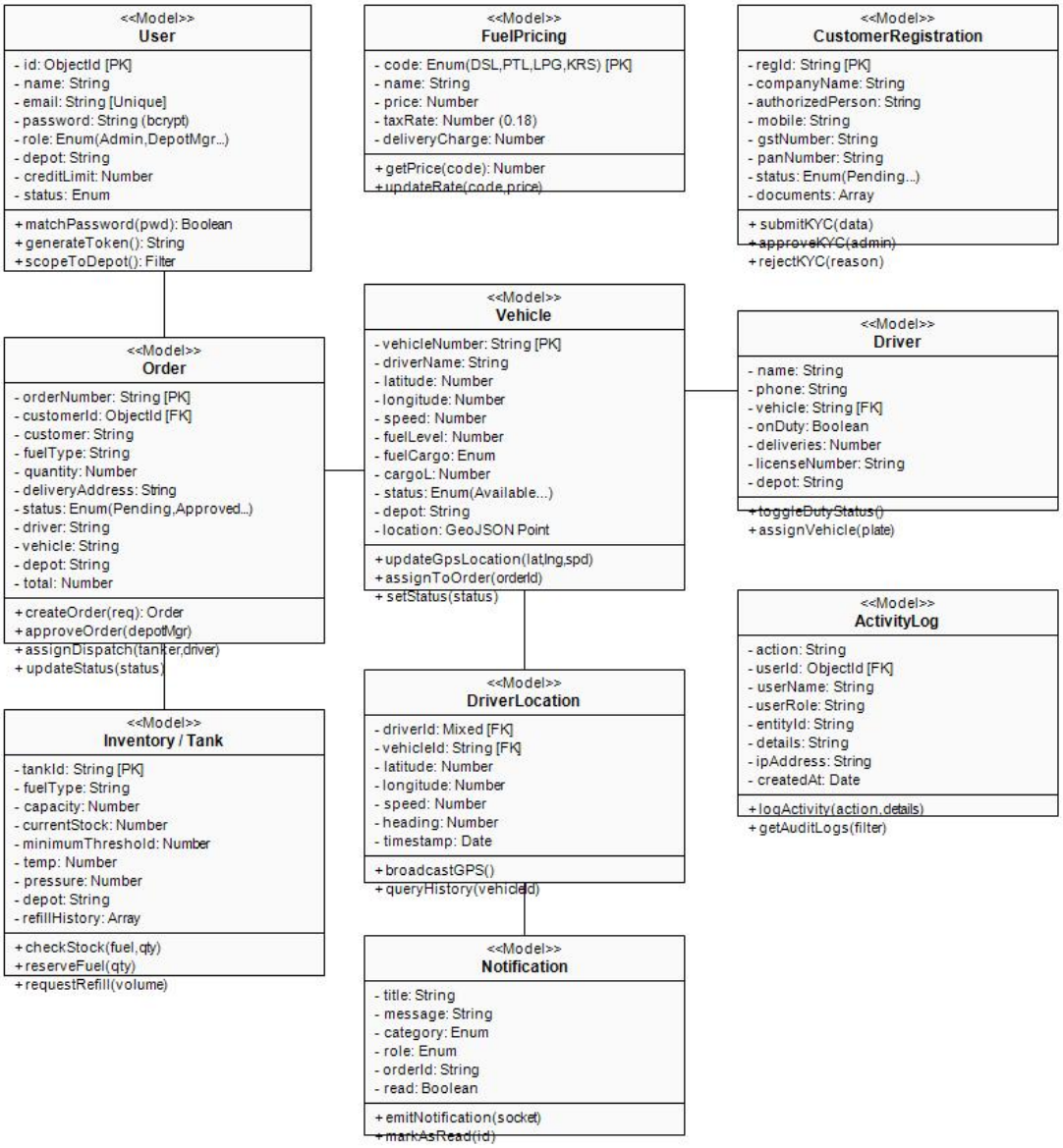


Fig 4.1 UML Class Diagram of Smart Fuel Dispatch & Fleet Telemetry System

CHAPTER 5
TABLE STRUCTURES

Users

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	trim
email	varchar(100)	NO	UNI	NULL	lowercase, indexed
password	varchar(255)	NO		NULL	bcrypt hash
role	enum('Admin','Depot Manager','Customer','Driver','Support Executive','Auditor')	NO		'Customer'	RBAC role
site	varchar(255)	YES		''	Delivery site
city	varchar(100)	YES		''	City jurisdiction
depot	varchar(50)	YES	MUL	''	Foreign key to Depot
creditLimit	decimal(12,2)	YES		0.00	Credit allowance
creditUsed	decimal(12,2)	YES		0.00	Consumed credit
phone	varchar(30)	YES		''	Contact telephone
isActive	tinyint(1)	NO		1	Active status toggle
status	enum('Pending','Approved','Rejected','Active','Inactive')	NO		'Active'	Account status

Field	Type	Null	Key	Default	Extra
loginAttempts	int	NO		0	Rate limiting counter
createdAt	datetime	NO		CURRENT_TIMESTAMP	
updatedAt	datetime	NO		CURRENT_TIMESTAMP	

Vehicles

Field	Type	Null	Key	Default	Extra
vehicleNumber	varchar(30)	NO	PRI	NULL	RTO plate number
driverName	varchar(100)	YES		'Unassigned'	Assigned driver
latitude	decimal(10,6)	NO		13.0827	Current GPS latitude
longitude	decimal(10,6)	NO		80.2707	Current GPS longitude
speed	int	NO		0	Current speed (km/h)
fuelLevel	int	NO		100	Tanker fuel gauge %
status	enum('Available','Assigned','In Transit','Delivered','Idle','Dispatched','Delivering','Maintenance')	NO		'Available'	Vehicle state
fuelCargo	enum('DSL','PTL','LPG','KRS','None')	NO		'DSL'	Dedicated cargo

Field	Type	Null	Key	Default	Extra
cargoL	int	NO		0	Current volume (L)
tirePressure	int	YES		110	Sensor reading PSI
engineTemp	int	YES		85	Sensor reading Celsius
dest	varchar(255)	YES		'Depot Base'	Destination site
depot	varchar(50)	NO	MUL	'DEP-CHN-01'	Assigned terminal
location	Point	NO	SPATIAL	NULL	2dsphere GeoJSON
createdAt	datetime	NO		CURRENT_TIMESTAMP	
updatedAt	datetime	NO		CURRENT_TIMESTAMP	

Drivers

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	Driver legal name
phone	varchar(30)	NO		NULL	Mobile telephone
vehicle	varchar(30)	YES	MUL	''	Assigned tanker plate
onDuty	tinyint(1)	NO		1	Roster duty toggle

Field	Type	Null	Key	Default	Extra
deliveries	int	NO		0	Completed trip count
licenseNumber	varchar(50)	YES		''	Hazmat license ID
email	varchar(100)	YES		''	Login username
depot	varchar(50)	NO	MUL	'DEP-CHN-01'	Home depot terminal
createdAt	datetime	NO		CURRENT_TIMESTAMP	
updatedAt	datetime	NO		CURRENT_TIMESTAMP	

Tanks / Inventory

Field	Type	Null	Key	Default	Extra
tankId	varchar(50)	NO	PRI	NULL	Unique tank code
name	varchar(100)	NO		NULL	Display name
fuelType	varchar(20)	NO	MUL	NULL	Commodity code (DSL/PTL)
capacity	int	NO		NULL	Maximum volume (L)
currentStock	int	NO		NULL	Available volume (L)
minimumThreshold	int	NO		5000	Reorder alert level (L)
reserved	int	NO		0	Locked for active orders
temp	decimal(5,2)	YES		25.0	Temperature sensor (C)
pressure	decimal(5,2)	YES		1.0	Pressure sensor (bar)
depot	varchar(50)	NO	MUL	'DEP-CHN-01'	Terminal code
lastRefill	varchar(30)	YES		CURDATE()	Latest refill date

Field	Type	Null	Key	Default	Extra
createdAt	datetime	NO		CURRENT_TIMESTAMP	
updatedAt	datetime	NO		CURRENT_TIMESTAMP	

Orders

Field	Type	Null	Key	Default	Extra
orderNumber	varchar(50)	NO	PRI	NULL	ORD-YYMMDD-XXX format
customerId	ObjectId	YES	MUL	NULL	Foreign key to Users
customer	varchar(150)	NO		'Customer'	Company trade name
fuelType	varchar(20)	NO		NULL	Fuel code (DSL, PTL...)
quantity	int	NO		NULL	Volume in Litres
deliveryAddress	text	NO		NULL	Site delivery location
status	enum('Pending','Approved','Rejected','Assigned','In Transit','Dispatched','Delivered','Cancelled')	NO		'Pending'	Workflow state
driver	varchar(100)	YES		'Unassigned'	Assigned driver name
vehicle	varchar(30)	YES		'Unassigned'	Assigned tanker plate
city	varchar(100)	YES		'Chennai'	Delivery city
depot	varchar(50)	NO	MUL	'DEP-CHN-01'	Fulfilling terminal

Field	Type	Null	Key	Default	Extra
slot	varchar(50)	YES		'08:00-10:00'	Delivery time window
subtotal	decimal(12,2)	NO		0.00	Base fuel charge
tax	decimal(12,2)	NO		0.00	18% GST (CGST+SGST)
deliveryCharge	decimal(12,2)	NO		250.00	Hazardous freight fee
total	decimal(12,2)	NO		0.00	Grand total (INR)
deliveryProof	text	YES		''	Discharge meter reading
customerSignature	text	YES		''	Digital signature
createdAt	datetime	NO		CURRENT_TIMESTAMP	
updatedAt	datetime	NO		CURRENT_TIMESTAMP	

FuelPricing

Field	Type	Null	Key	Default	Extra
code	enum('DSL','PTL','LPG','KRS','HSD','MS','JET-A1','BIO-D')	NO	PRI	NULL	Commodity symbol
name	varchar(100)	NO		NULL	Official product title
price	decimal(10,2)	NO		NULL	Base rate per Litre

Field	Type	Null	Key	Default	Extra
color	varchar(20)	YES		'#f59e0b'	Hex UI color code
taxRate	decimal(5,2)	NO		0.18	GST statutory rate
deliveryCharge	decimal(10,2)	NO		250.00	Delivery surcharge
updatedAt	datetime	NO		CURRENT_TIMESTAMP	Price revision time

CustomerRegistrations

Field	Type	Null	Key	Default	Extra
regId	varchar(50)	NO	PRI	NULL	REG-YYYY-XXXX
email	varchar(100)	NO	UNI	NULL	Contact email address
companyName	varchar(150)	NO		NULL	Registered enterprise
authorizedPerson	varchar(100)	NO		NULL	Authorized executive
mobile	varchar(30)	NO		NULL	Verified mobile phone
gstNumber	varchar(30)	NO		NULL	15-digit GSTIN
panNumber	varchar(20)	NO		NULL	10-digit Income Tax PAN
address	text	NO		NULL	Factory / site address

Field	Type	Null	Key	Default	Extra
city	varchar(100)	NO		'Chennai'	District location
state	varchar(50)	NO		'Tamil Nadu'	Jurisdiction state
pincode	varchar(10)	NO		'600001'	Postal index code
status	enum('Pending','Approved','Rejected','Pending Review')	NO		'Pending'	Administrative approval
approvedBy	varchar(100)	YES		NULL	Admin approver name
submittedAt	datetime	NO		CURRENT_TIMESTAMP	Submission timestamp

ActivityLogs

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
action	varchar(100)	NO	MUL	NULL	Event classification
userId	ObjectId	YES	MUL	NULL	Executing user account
userName	varchar(100)	YES		'System'	Actor full name
userRole	varchar(50)	YES		'Admin'	Role at execution
entityId	varchar(100)	YES		''	Target order / vehicle ID
details	text	YES		''	Audit event description
ipAddress	varchar(50)	YES		''	Client IP address
createdAt	datetime	NO	MUL	CURRENT_TIMESTAMP	Immutable log time

DriverLocations

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
driverId	Mixed	YES	MUL	NULL	Driver identification
driverName	varchar(100)	YES		''	Driver name
vehicleId	varchar(50)	NO	MUL	NULL	Tanker plate number
latitude	decimal(10,6)	NO		NULL	GPS latitude reading
longitude	decimal(10,6)	NO		NULL	GPS longitude reading
speed	int	NO		0	Speed telemetry (km/h)
heading	int	YES		0	Compass heading (deg)
accuracy	int	YES		0	GPS precision radius (m)
timestamp	datetime	NO	MUL	CURRENT_TIMESTAMP	Telemetry broadcast time

Notifications

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
title	varchar(150)	NO		NULL	Alert subject line
message	text	NO		NULL	Notification content
category	enum('order','delivery','inventory','registration','general','request','dispatch')	NO		'general'	Domain topic
role	enum('Admin','Depot Manager','Customer','Driver','All')	NO		'All'	Recipient role target
type	enum('info','success','warning','danger')	NO		'info'	Severity

Field	Type	Null	Key	Default	Extra
					level
orderId	varchar(50)	YES		''	Related consignment
read	tinyint(1)	NO		0	Read status indicator
createdAt	datetime	NO		CURRENT_TIMESTAMP	Alert creation time

Otp

Field	Type	Null	Key	Default	Extra
_id	ObjectId	NO	PRI	NULL	auto_increment
email	varchar(100)	YES	MUL	NULL	Associated email
phone	varchar(30)	YES	MUL	NULL	Associated phone number
otp	varchar(10)	NO		NULL	Cryptographic OTP string
type	enum('login','reset','verification')	NO		'login'	Security purpose
expiresAt	datetime	NO		NULL	Expiration timestamp
verified	tinyint(1)	NO		0	Verification status
attempts	int	NO		0	Verification attempt counter

CHAPTER 6
ER DIAGRAM

ADDITIONAL MASTER TABLES

The master data architecture defines foundational reference datasets and enterprise catalogs across the fuel distribution network. Master tables such as FuelPricing, Regional Terminal Depots, Notifications, and OTP verification records provide unified lookup standards across all transactional operations, preventing redundancy and enforcing administrative consistency.

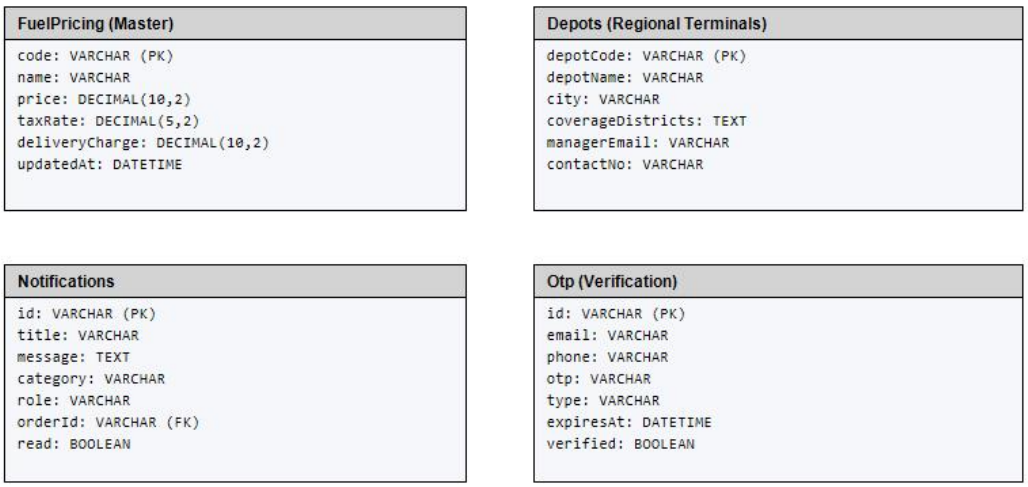


Fig 6.1 Additional Master Tables Schema

The comprehensive Entity-Relationship (ER) Diagram illustrates the complete logical data model of the Smart Fuel Dispatch & Fleet Telemetry System. It details entity attributes, primary keys, foreign key references, and relational cardinalities between core domain entities including Users, Regional Depots, Storage Tanks, Calibrated Vehicles, Drivers, Orders, DriverLocations, and ActivityLogs.

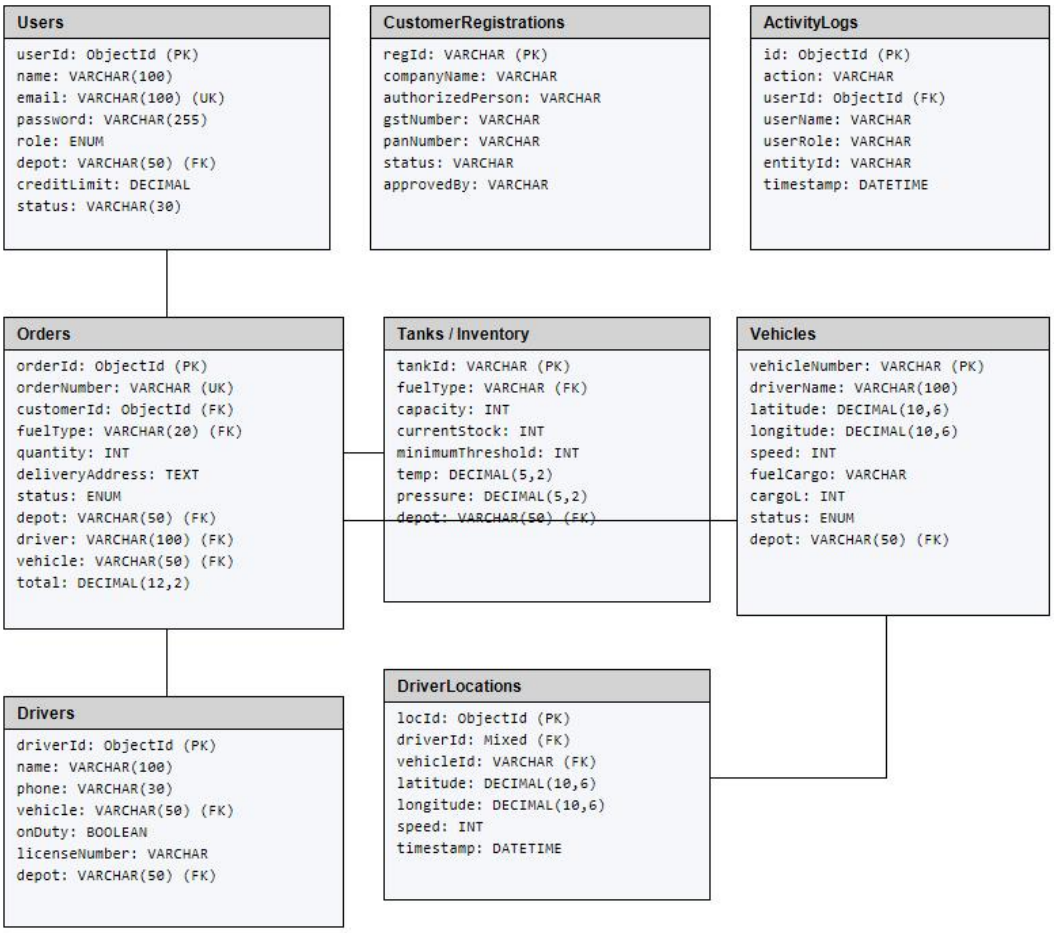


Fig 6.2 Entity-Relationship Diagram of Smart Fuel Dispatch & Fleet Telemetry System

CHAPTER 7
TECH STACK

7.1 TECH STACK

Layer	Technology	Purpose
Frontend	React.js (Vite)	Single-page application with dynamic dispatch UI and live state updates
Styling & UI	TailwindCSS + Lucide Icons	Utility-first design system, responsive dashboards, and interactive widgets
State Management	React Context API + useReducer	Centralized state machines for Authentication, Orders, Inventory, and Settings
HTTP Client	Axios	REST API communication with automated JWT Bearer auth and 401 interceptors
Real-Time Telemetry	Socket.IO (WebSockets)	Bidirectional low-latency event broadcasting for live GPS tracking and alerts
GIS Mapping	Leaflet.js / OpenStreetMap	Interactive geographical map rendering with tanker markers and route polyline breadcrumbs
Backend	Node.js + Express.js v5	Modular, non-blocking RESTful API server with robust error middleware
ORM / ODM	Mongoose v8	Object modeling, schema constraint validation, lifecycle hooks, and virtuals
Database	MongoDB Atlas	Document storage with 2dsphere spatial indexing for geospatial telemetry queries
Authentication	JSON Web Tokens (JWT) + bcryptjs	Stateless, token-based authentication and secure salted password hashing
Security	CORS, Rate Limiting, Helmet	HTTP header protection, IP-based brute-force defence, and terminal scoping
File Uploads	Multer	Secure multipart form-data handling for corporate KYC certificate uploads
CI/CD	Jenkins (Jenkinsfile)	Automated continuous integration, test execution, and production build pipelines

7.2 ARCHITECTURE OVERVIEW

Layer	Technology	Purpose
Model	Mongoose ODM + MongoDB Atlas	Defines document schemas, enforces validation rules, manages indexes, and encapsulates data persistence
View	React.js + Context API	Renders the dynamic graphical user interface, manages user inputs, and communicates with backend via Axios
Controller	Node.js + Express.js v5	Receives HTTP requests, authenticates tokens, applies business logic and depot scoping, and coordinates Model-View flow

CHAPTER 8
MODULES

8.0 Design Approach

Each module of the Smart Fuel Dispatch & Fleet Telemetry System is described here the way a comprehensive enterprise system design problem is solved: by walking through the same seven architectural steps in the same order, every time. This ensures the technical documentation of every module remains mathematically rigorous, uniform, and comparable, establishing why each architectural trade-off was adopted rather than merely presenting graphical interface elements.

Step	Question it answers	Presented in each module as
1. Functional requirements	What specific operational capabilities must the module provide?	Section x.1
2. Non-functional requirements	How fast, how reliable, and how resilient? Where does the CAP trade-off lie?	Section x.2
3. Tech stack	Which architectural libraries and tools address this domain, and why?	Section x.3
4. Back-of-the-envelope	What are the data volumes, request throughputs, and storage capacities?	Section x.4
5. Low-level architecture	Which software components interact and what is the end-to-end request flow?	Section x.5 (diagram + flow)
6. API design	What is the RESTful API / WebSocket protocol contract between client and server?	Section x.6
7. DB design	Which MongoDB collections, schema constraints, and indexes persist the data?	Section x.7

Shared baseline for all modules

Non-functional baseline: Every client request is intercepted by CORS, Helmet (HTTP header security), JSON body parsers, IP rate limiting, and JWT authentication middleware before reaching domain controllers. All persistent data resides in MongoDB Atlas with replica sets, prioritizing consistency over availability (CP in CAP theorem) for mission-critical financial, tariff, and inventory updates: an operator cannot proceed with dispatch if the central database is unreachable, guaranteeing that two concurrent depot managers can never over-commit stock from the same storage vault or assign conflicting consignments to the same tanker. Financial amounts use rounded two-decimal precision, preventing floating-point arithmetic drift.

Sizing baseline (assumptions)

Quantity	Estimate	Basis
Named users	~ 50 active accounts	Depot staff, drivers, support agents, corporate clients
Operating days / year	365 days / year	Continuous 24x7 industrial fuel supply operations
Requests per user / day	~ 400 requests	Dashboard loads, live GPS pings, order status updates
Total requests per day	50 x 400 = 20,000	Derived daily transactional throughput
Average load	20,000 / 86,400 s ~ 0.23 req/s	Distributed throughout 24-hour delivery schedule
Peak load	~ 5-10 req/s	Morning dispatch batch and shift transitions
Database collections	12 active collections	Live MongoDB Atlas schema

A single Node.js runtime process and a MongoDB Atlas M0/M10 cluster comfortably absorb this workload. The stateless JWT authentication architecture allows additional Node.js microservices to scale horizontally behind a reverse proxy as regional depot volume expands.

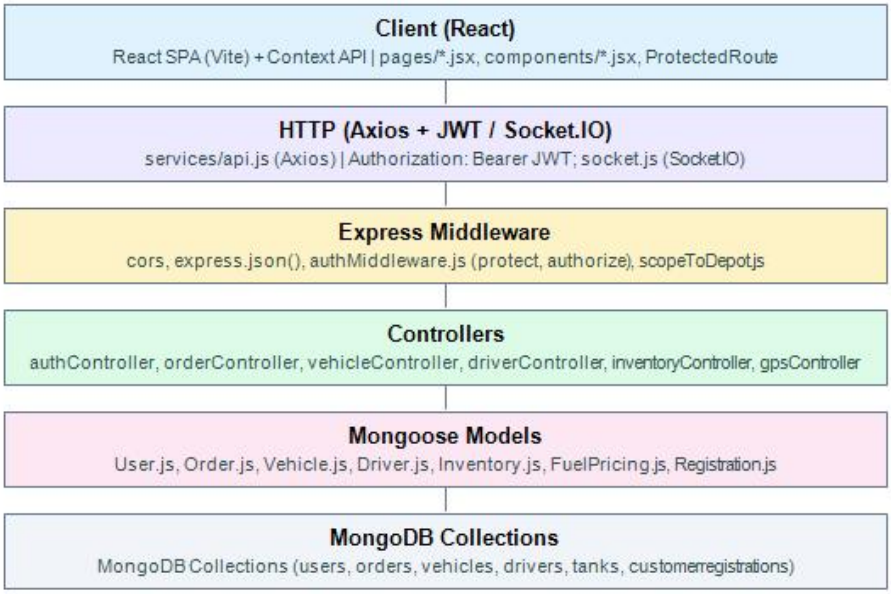


Fig 8.0 Overall request flow of Smart Fuel Dispatch & Fleet Telemetry System

Fig 8.0 Overall request flow of the Smart Fuel Dispatch & Fleet Telemetry System

8.1 FOLDER STRUCTURE

```
fuel-delivery-app (Workspace Root)
 v fuel-delivery-app
   > .git
   > backend
   > dist
   > node_modules
   > public
   > src
     .env
     Jenkinsfile
     README.md
     ROLES.md
     package.json
     vite.config.js
```

Fig 8.1.1 Highlevel view of project

8.1.1 BACKEND FOLDER

```
backend / Overall Architecture
 v backend
   > config
   > controllers
   > middleware
   > models
   > node_modules
   > routes
   > uploads
   > utils
     .env
     package.json
     seeder.js
     server.js
```

Fig 8.1.2 Backend Folder Structure - Overall organization of backend files and directories showing modular architecture

```
backend / config
 v backend
   v config
       db.js (MongoDB Mongoose connection and pool)
       socket.js (Socket.IO Real-time WebSockets)
```

Fig 8.1.3 config Folder structure

```
backend / controllers
 v backend
   v controllers
       activityController.js
       authController.js
       customerController.js
       driverController.js
       gpsController.js
       inventoryController.js
       notificationController.js
       orderController.js
       pricingController.js
       registrationController.js
       reportController.js
       vehicleController.js
```

Fig 8.1.4 Controller Folder

```
backend / middleware
 v backend
   v middleware
       authMiddleware.js (JWT verify and RBAC roles)
       errorMiddleware.js (404 and centralized error handler)
       permissions.js (Granular operational permissions)
```

Fig 8.1.5 Middleware Folder Structure

```
backend / models
 v backend
   v models
       ActivityLog.js
       Customer.js
       Driver.js
       DriverLocation.js
       FuelPricing.js
       Inventory.js
       Notification.js
       Order.js
       Otp.js
       Registration.js
       User.js
       Vehicle.js
       VehicleLocation.js
```

Fig 8.1.6 Model Folder Structure

8.1.2 FRONTEND FOLDER



Fig 8.1.7 Structure of Frontend Folder

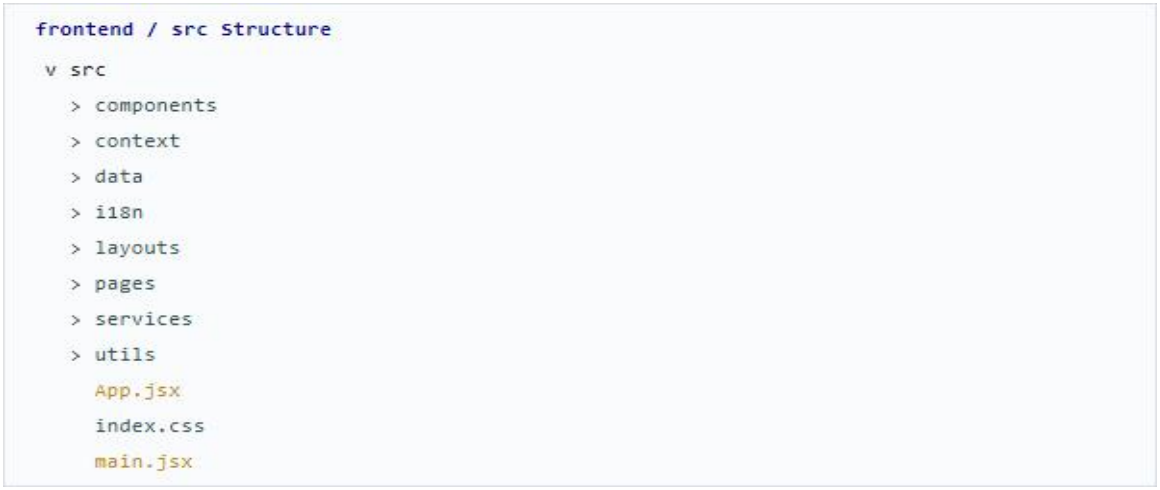


Fig 8.1.8 Structure of frontend/src

```
frontend / src / pages
 v src
   v pages
       Applications.jsx
       AuditLogs.jsx
       CustomerPortal.jsx
       Customers.jsx
       Dashboard.jsx
       DriverPortal.jsx
       Drivers.jsx
       FleetMap.jsx
       ForgotPassword.jsx
       Inventory.jsx
       Login.jsx
       OrderDetail.jsx
       Orders.jsx
       PendingRequests.jsx
       Pricing.jsx
       Register.jsx
       Reports.jsx
       Settings.jsx
       UserManagement.jsx
       Vehicles.jsx
```

Fig 8.1.9 Structure of frontend/src/pages Folder

```
frontend / src / routes and services
 v src
   v services
       api.js (Axios HTTP instance and REST clients)
       recentAccess.js (Browser navigation cache)
       socket.js (Socket.IO client telemetry stream)
   v layouts
       AppLayout.jsx (Header, Sidebar, Shell)
```

Fig 8.1.10 Structure of frontend/src/routes and frontend/src/services

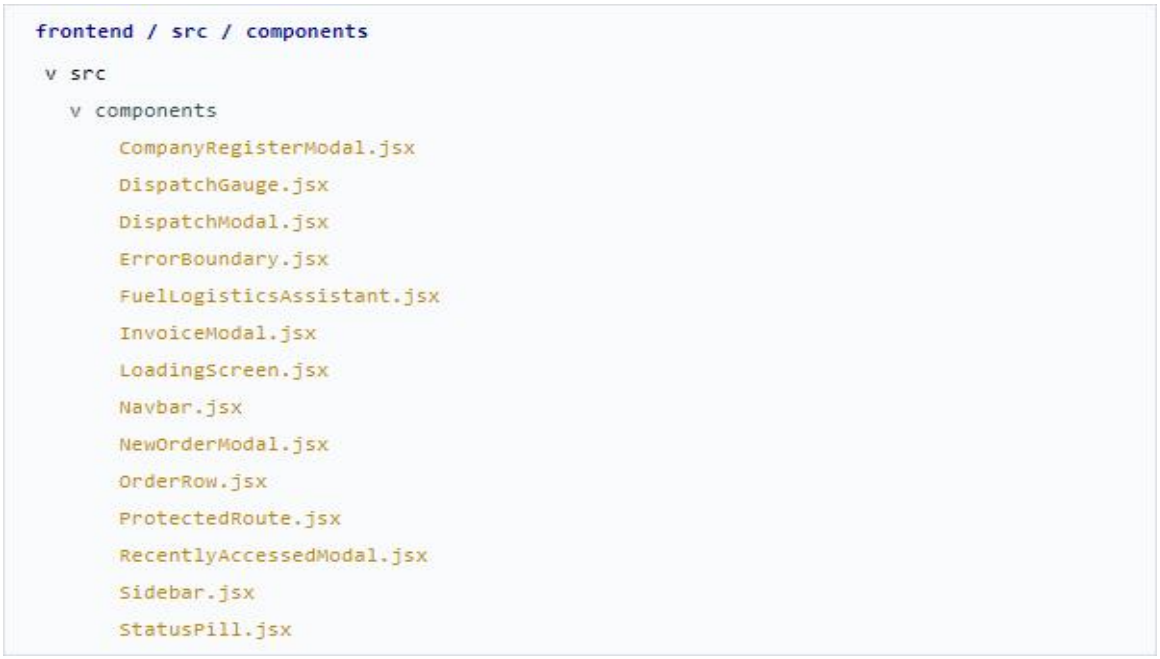


Fig 8.1.11 Structure of frontend/src/components

8.2 USER AUTHENTICATION & ACCESS CONTROL

8.2.0 Purpose & Scope

Provides tokenized authentication, salted credential protection, session management, and granular Role-Based Access Control (RBAC) across the enterprise logistics network. Enforces terminal jurisdictional boundaries to restrict depot managers to their assigned regional hub while empowering administrators with global oversight and granting customers and drivers dedicated self-service portals.

8.2.1 Functional Requirements

What the module must do:

- A user authenticates with email/username and password or one-time passcode (OTP) and receives a signed JSON Web Token (JWT) along with an active user profile (id, name, email, role, depot, creditLimit).
- Provides one-click enterprise demo account quick-switch buttons for all 6 roles (Admin, Chennai Depot Mgr, Coimbatore Depot Mgr, Support, Auditor, Customer, Driver).
- Enforces 6 distinct RBAC roles deciding permissible API routes: Admin, Depot Manager, Support Executive, Auditor, Customer, and Driver.
- Terminal Jurisdiction Isolation: Attaches depot scope ('DEP-CHN-01', 'DEP-CBE-05', etc.) to staff profiles, automatically filtering queries via scopeToDepot middleware.
- Allows users to securely reset forgotten passwords via OTP email verification (forgotPassword and verifyOTP endpoints).
- Protects client routes via ProtectedRoute.jsx, redirecting unauthenticated sessions to /login and intercepting HTTP 401 token expiration.

Out of scope: biometric facial scanning, physical smart-card hardware readers.

8.2.2 Non-Functional Requirements

How well the module must do it:

- Security: Passwords hashed with bcryptjs using auto-generated salt rounds; cryptographic hashes are excluded from API payloads by default.
- Brute-force defence: Limits consecutive login attempts per IP address, locking accounts after 5 repeated invalid submissions.
- Session validity: Tokens are signed with HMAC-SHA256 and expire after 12 hours, balancing operational shift convenience with security.
- Stateless scalability: Node.js server instances validate JWT signatures independently without querying a shared server-side session cache.
- Latency target: Login authentication completes within 300 ms, with signature verification taking under 5 microseconds per request.
- CAP consistency: The MongoDB users collection acts as the single source of truth; deactivated accounts are immediately rejected upon subsequent token verification.

8.2.3 Tech Stack

Concern	Choice	Why it fits this module

Concern	Choice	Why it fits this module
Password hashing	bcryptjs (salted)	Cryptographically secure key derivation without native C++ compilation dependencies
Session token	JSON Web Token (JWT, 12 h)	Stateless bearer authentication passed via HTTP Authorization header
Terminal scoping	scopeToDepot utility	Enforces multi-tenant data isolation across regional hubs at the database query level
Route guard	ProtectedRoute.jsx + Axios	Client-side route interception with automated redirect upon 401 Unauthorized status
OTP verification	Crypto OTP generator	Enables secure out-of-band credential reset and emergency mobile sign-in

8.2.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Registered users	~ 50 accounts, < 50 KB	Staff, drivers, customers (~ 1 KB per account document)
Logins per day	~ 100 logins	50 active users x 2 shift logins (12-hour expiration)
Login burst	~ 25 logins in 10 mins	Morning terminal shift commencement (~ 0.04 logins/s)
CPU per login	~ 80 ms	Bcrypt hash evaluation; single CPU core handles > 12 logins/s
Authenticated API calls	~ 20,000 / day	Each request cryptographically verifies the JWT signature
Token payload size	~ 450 bytes	Compact header, payload (id, role, depot), and signature

8.2.5 Low-Level Architecture

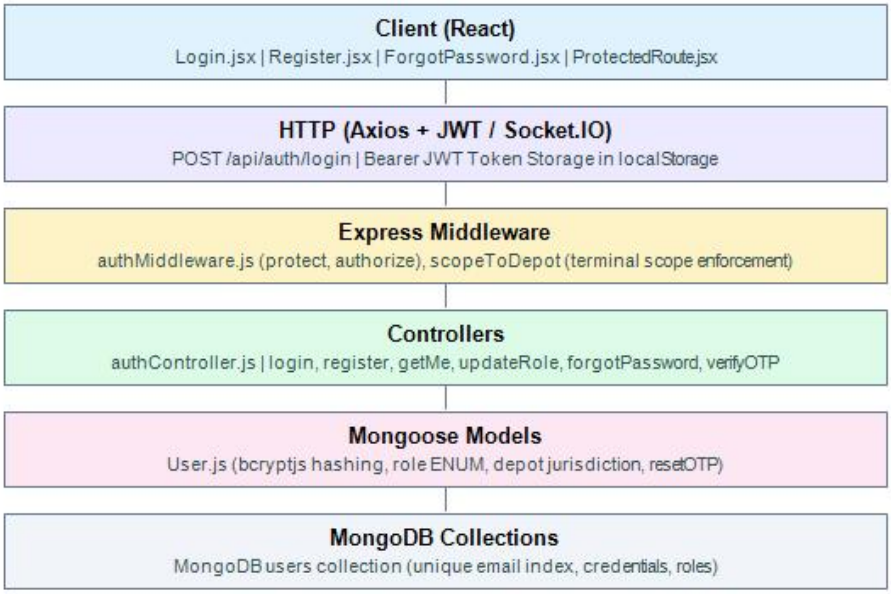


Fig 8.2.A Low-level architecture - USER AUTHENTICATION & ACCESS CONTROL

Fig 8.2.A Low-level architecture - USER AUTHENTICATION & ACCESS CONTROL

Request flow: * Login.jsx submits user credentials via POST /api/auth/login to the Express server. * authController looks up the user document by email/username in MongoDB, checks that the account is active, and verifies the password using bcrypt.compare(). * Upon successful authentication, last_login is updated, a signed JWT token is issued, and the sanitized user profile (including role and assigned depot) is returned. * The client stores the token in localStorage; services/api.js automatically attaches the Bearer token to all outgoing HTTP requests. * Backend middleware (authMiddleware.js and scopeToDepot.js) verifies the token signature, checks role permissions, and injects depot query filters before handing execution to the controller. * If an expired or invalid token is received, the server answers HTTP 401, triggering the Axios response interceptor to clear local storage and route the user to /login.

8.2.6 API Design

Method	Endpoint	Purpose / contract
POST	/api/auth/login	Public. Authenticates credentials and returns JWT bearer token + user profile
POST	/api/auth/register	Public / Customer. Registers new self-service customer account
GET	/api/auth/me	Protected. Returns current signed-in user profile, role, and depot assignment
POST	/api/auth/forgot-password	Public. Sends 6-digit OTP code to registered email for password recovery
POST	/api/auth/verify-otp	Public. Validates recovery OTP and authorizes

Method	Endpoint	Purpose / contract
		password change
GET	/api/auth/users	Admin only. Lists all registered staff, customer, and driver accounts
PATCH	/api/auth/users/:id/role	Admin only. Promotes or demotes user role within the RBAC matrix
PATCH	/api/auth/users/:id/depot	Admin only. Reallocates staff member to a designated regional terminal

8.2.7 DB Design

Table / Collection	Keys & indexes	Key columns
Users	PK _id (ObjectId) UNIQUE email INDEX depot, role	name, email, password (hash), role ('Admin','Depot Manager','Customer','Driver'...), depot ('DEP-CHN-01'...), creditLimit, isActive, status
Otp	PK _id (ObjectId) INDEX email, expiresAt	email, phone, otp, type ('login','reset'), expiresAt, verified, attempts

* The unique index on email accelerates login lookups to single index reads and prevents duplicate account registrations. * Terminal scoping is indexed on depot, ensuring sub-millisecond filtering across thousands of localized records.

8.2.8 Screenshots

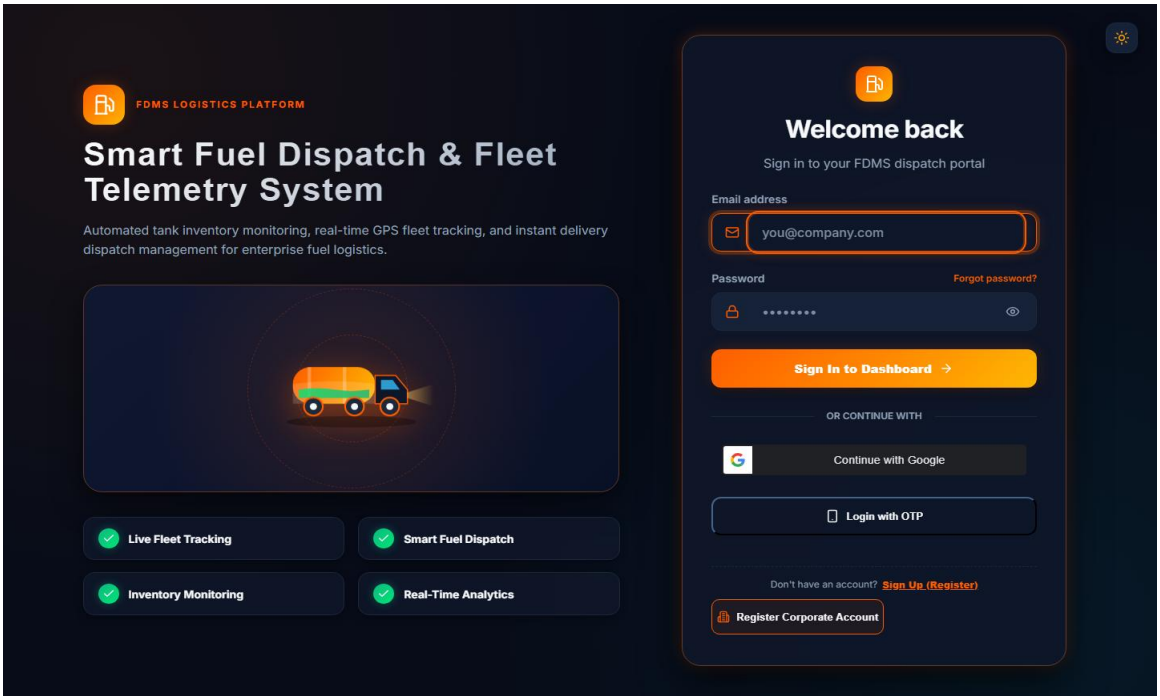


Fig 8.2 User Login & Terminal Authentication Interface

The authentication portal presents an intuitive single-sign-on interface with integrated security controls. Users can authenticate using their registered corporate email and password or one-time passcodes. For seamless demonstration, the interface includes interactive role chips (Admin, Chennai Depot Manager, Coimbatore Depot Manager, Driver, Auditor) that automatically populate valid enterprise credentials. The interface validates inputs on the client, enforces rate limiting, and securely receives the signed JWT session.

8.3 FLEET MANAGEMENT & TANKER VEHICLES

8.3.0 Purpose & Scope

Unifies foundational logistics assets and enterprise registers that establish the baseline for all fuel dispatch operations across Tamil Nadu. Manages calibrated delivery tanker vehicles, specialized multi-compartment fuel allocations, driver personnel rosters with hazardous material transport certifications, wholesale fuel price tariffs, regional terminal hubs, and corporate customer KYC profiles.

8.3.1 Functional Requirements

What the module must do:

- Tanker Fleet Registry: Create, inspect, edit, and decommission delivery tankers with plate number, dedicated cargo type (DSL, PTL, LPG, KRS), volume capacity, and terminal depot assignment.
- Onboard and maintain delivery drivers with contact mobile numbers, hazardous material transport licenses, assigned delivery vehicles, and active trip counters.
- On-Duty Status Management: Provide instant on-duty / off-duty toggle switches allowing depot managers to dynamically adjust the dispatch-ready driver pool.
- Wholesale Tariff Schedules: Centrally define official wholesale fuel tariffs, statutory GST percentages (18%), and fixed hazardous delivery freight surcharges.
- Regional Terminal Depots: Maintain authoritative terminal codes (DEP-CHN-01, DEP-SLM-02, DEP-MDU-03, DEP-TRY-04, DEP-CBE-05) with district jurisdictional coverage.
- Corporate Customer Onboarding & KYC: Capture company identity, GSTIN, Income Tax PAN, uploaded incorporation certificates, and commercial credit limits.
- Multi-depot administrative isolation: Depot managers can add/manage fleet assets strictly within their terminal, while Platform Admins govern platform-wide tariffs and approvals.

8.3.2 Non-Functional Requirements

How well the module must do it:

- Read-heavy optimization: Master fleet and pricing data are accessed frequently during order calculation and dispatch (~ 50 reads per write); lookups complete in under 50 ms.
- Geospatial indexing: Delivery vehicle records maintain a 2dsphere index on GeoJSON location coordinates to support fast radius and proximity searches.
- Data integrity: Unique compound constraints on vehicle plates and driver license IDs prevent duplicate asset registration across depots.
- Role-based enforcement: Only Platform Admin can edit wholesale tariffs; Depot Managers have read-only visibility on tariffs but full CRUD on local tankers and drivers.
- CAP consistency: Updates to vehicle cargo and driver duty state commit synchronously, preventing conflicting assignments across concurrent dispatch requests.

8.3.3 Tech Stack

Concern	Choice	Why it fits this module

Concern	Choice	Why it fits this module
Fleet management UI	React pages + Modals	Interactive vehicle cards and driver duty roster tables with live search
Geospatial indexing	MongoDB 2dsphere index	Enables high-performance proximity queries for nearest-tanker heuristic dispatch
Document upload	Multer middleware	Handles multipart/form-data storage for commercial customer KYC certificates
Data persistence	Mongoose Vehicle & Driver models	Strict schema validation for calibrated volumes, license patterns, and statuses

8.3.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Fleet delivery vehicles	~ 50 tankers across 5 hubs	10 tankers per regional terminal (~ 2 KB per document)
Active drivers	~ 60 certified operators	1.2 drivers per vehicle for rotational shift coverage
Fuel pricing tariffs	4 primary commodities	HSD Diesel, MS Petrol, LPG, Industrial Kerosene (< 5 KB)
Regional depots	5 primary terminals	Chennai, Salem, Madurai, Trichy, Coimbatore
Vehicle query latency	P95 < 20 ms	In-memory indexed lookup by vehicleNumber or depot

8.3.5 Low-Level Architecture

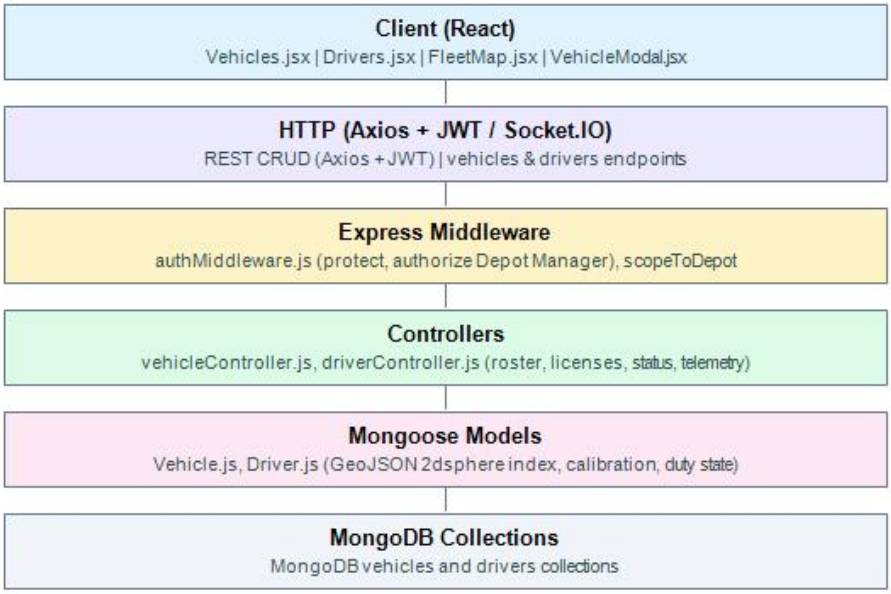


Fig 8.3.A Low-level architecture - FLEET MANAGEMENT & TANKER VEHICLES

Fig 8.3.A Low-level architecture - FLEET MANAGEMENT & TANKER VEHICLES

Request flow: * Vehicles.jsx / Drivers.jsx issue authenticated GET requests to /api/vehicles and /api/drivers with depot scope. * vehicleController and driverController apply scopeToDepot filter so depot managers only inspect assets belonging to their terminal. * Modal dialogs POST or PUT records; backend controllers validate calibrated capacities, RTO plate patterns, and unique license numbers before saving. * Real-time duty toggles emit Socket.IO broadcast events so dispatch queues reflect updated driver availability immediately.

8.3.6 API Design

Method	Endpoint	Purpose / contract
GET	/api/vehicles	Lists fleet tankers scoped to active depot with telemetry and status
POST	/api/vehicles	Depot Manager only. Registers a new delivery tanker vehicle
PUT	/api/vehicles/:id	Updates tanker specifications, fuel cargo compartment, and capacity
DELETE	/api/vehicles/:id	Admin only. Decommissions vehicle from active operations
GET	/api/drivers	Lists driver personnel with on-duty status and assigned vehicles
POST	/api/drivers	Depot Manager only. Onboards a new driver with hazmat credentials

Method	Endpoint	Purpose / contract
PATCH	/api/drivers/:id/duty	Toggles driver on-duty status for active dispatch eligibility
GET	/api/pricing	Public / Authenticated. Retrieves official wholesale fuel tariffs
PUT	/api/pricing/:code	Admin only. Updates base wholesale rate per litre for fuel commodity

8.3.7 DB Design

Table / Collection	Keys & indexes	Key columns
Vehicles	PK vehicleNumber 2DSPHERE location INDEX depot, status	vehicleNumber, driverName, latitude, longitude, speed, fuelCargo ('DSL','PTL'...), cargoL, status, depot
Drivers	PK _id INDEX depot, onDuty	name, phone, vehicle (FK), onDuty, deliveries, licenseNumber, email, depot
FuelPricing	PK code	code ('DSL','PTL','LPG','KRS'), name, price, taxRate, deliveryCharge, updatedAt

8.3.8 Screenshots

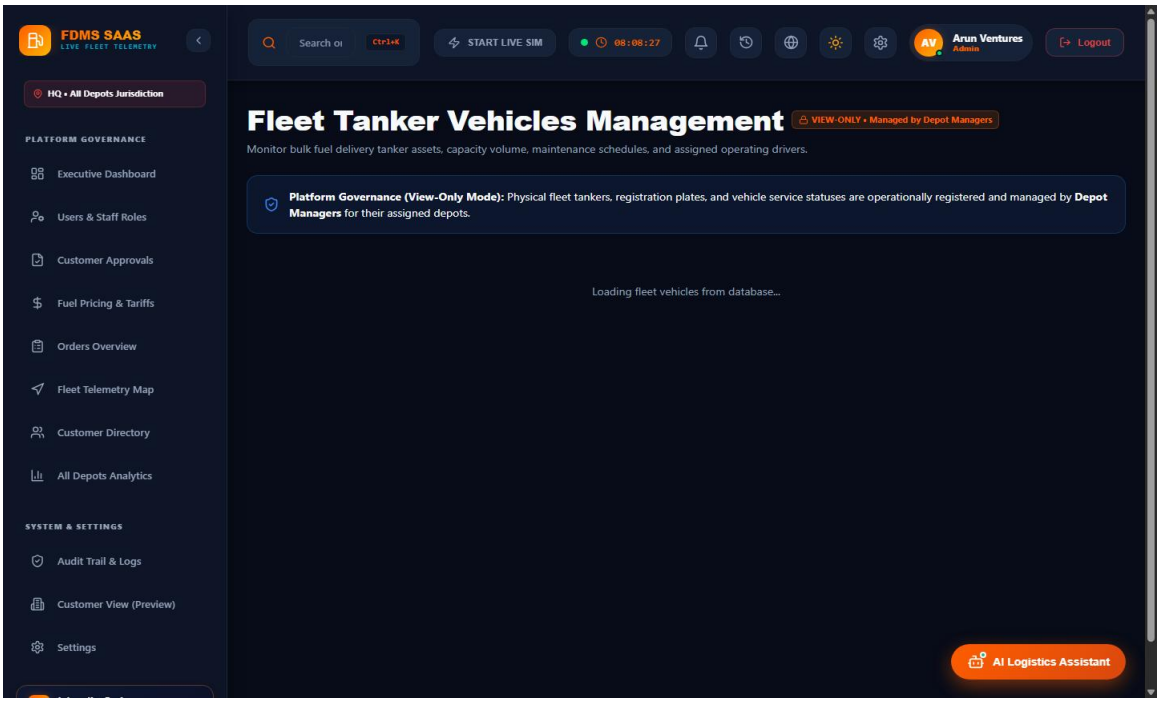


fig 8.3.1 Delivery tanker fleet registry

The delivery tanker fleet registry displays all vehicles registered under the terminal depot. The table summarizes plate numbers, assigned drivers, dedicated fuel cargos, calibrated payload capacities, current fuel levels, tire pressure readings, engine temperatures, and live operational statuses. Search and filter tools assist operators in locating tankers quickly.

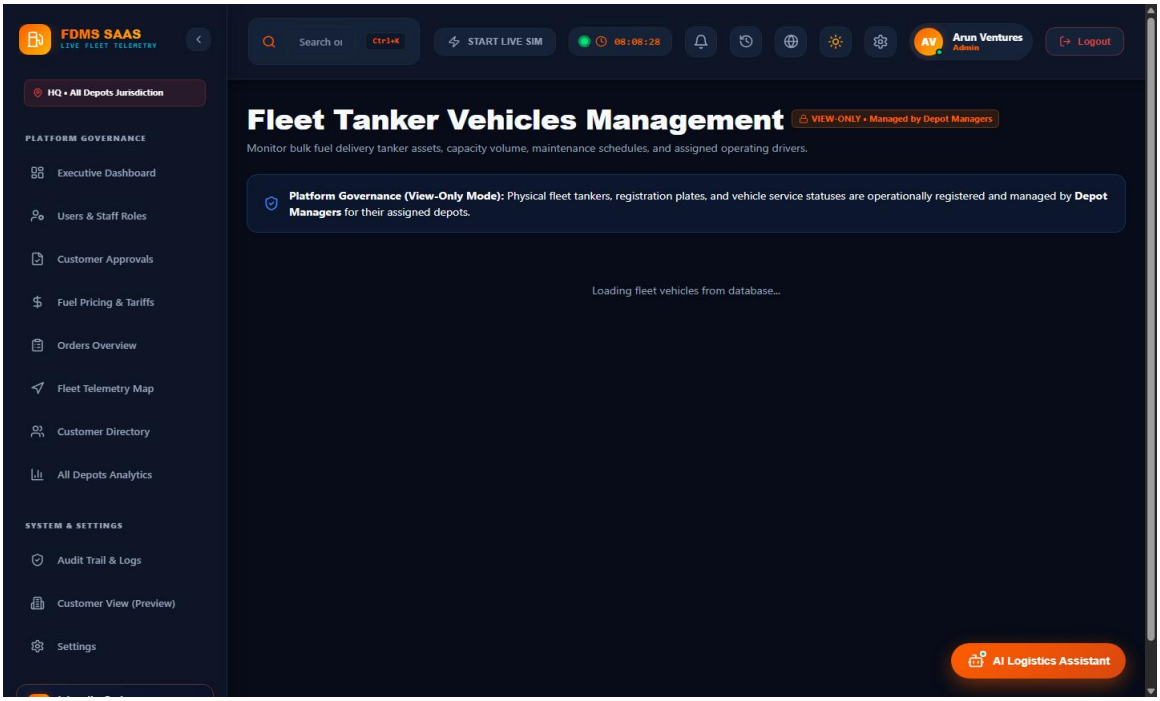


fig 8.3.2 Editing tanker specifications

The tanker specification dialog enables depot managers to modify calibrated volume parameters, assign designated drivers, and update dedicated fuel cargo types. Maintaining these calibration values ensures that dispatch algorithms allocate tankers strictly within legal payload limits.

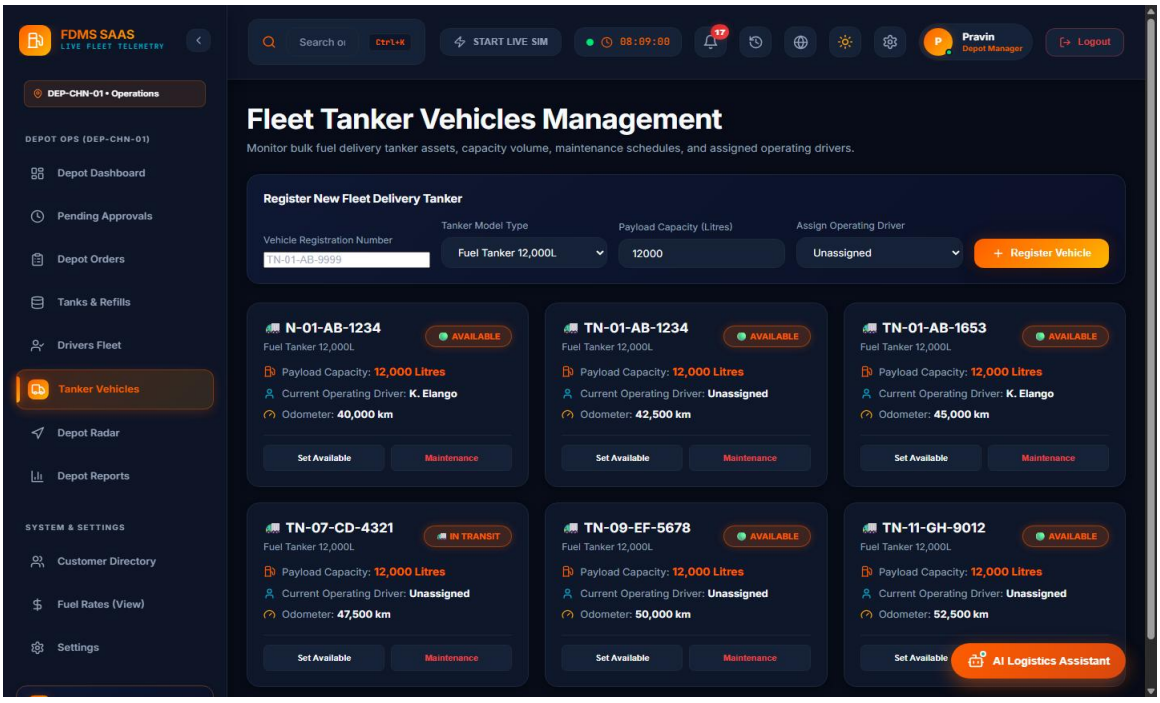


fig 8.3.3 Registering a new tanker vehicle

The vehicle onboarding form registers newly acquired tankers into the terminal fleet. Operators input official RTO registration plates, dedicated hazardous cargo compartments (DSL, PTL, LPG, KRS), PESO-calibrated tank capacities, and terminal depot allocations before committing the record to MongoDB.

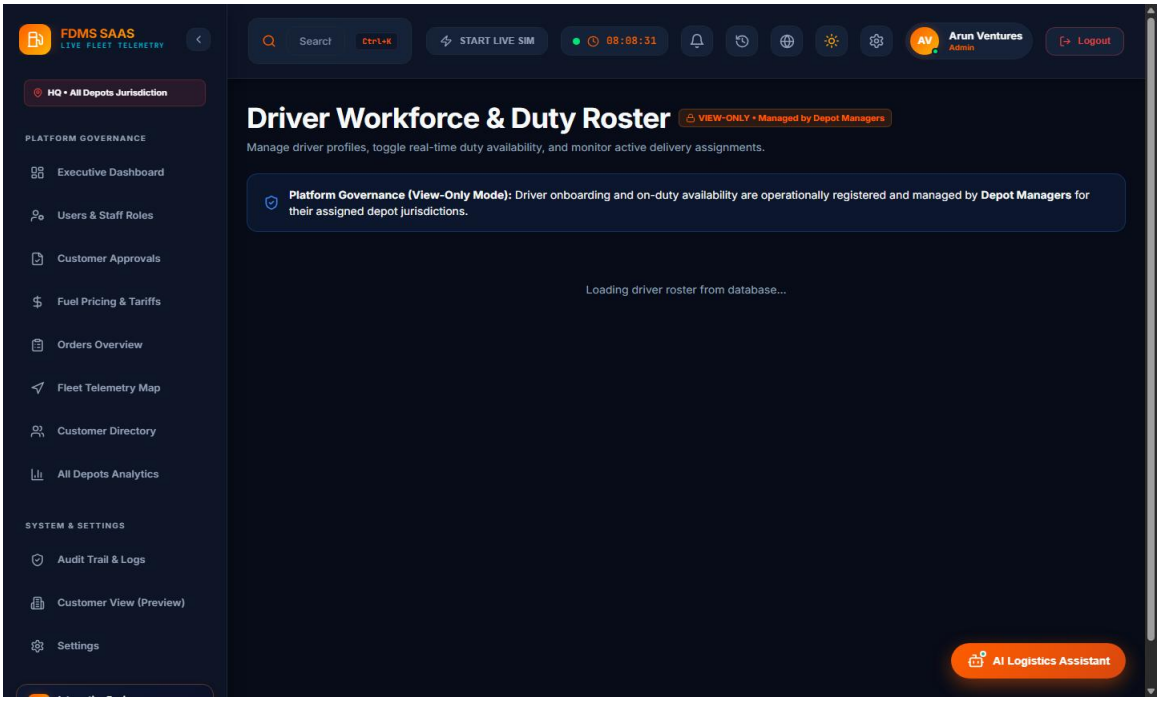


fig 8.3.4 Driver fleet roster & licensing

The driver fleet registry maintains verified personnel data across all terminal operators. The interface details driver names, contact mobile numbers, assigned tanker plates, statutory hazardous chemical transport license numbers, completed delivery counters, and active duty status indicators.

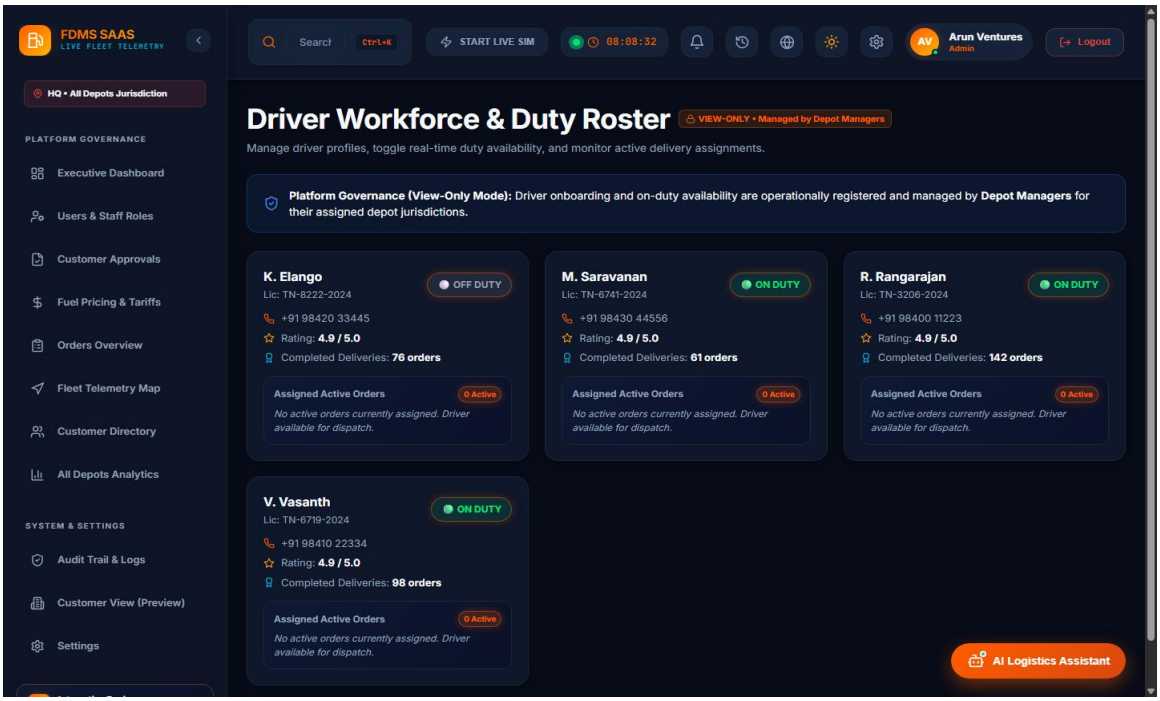


fig 8.3.5 Filtering drivers by on-duty status

The driver roster filter restricts the displayed list to drivers currently toggled On-Duty. Filtering ensures depot managers can immediately ascertain active personnel capacity during shift scheduling and emergency fuel dispatch operations.

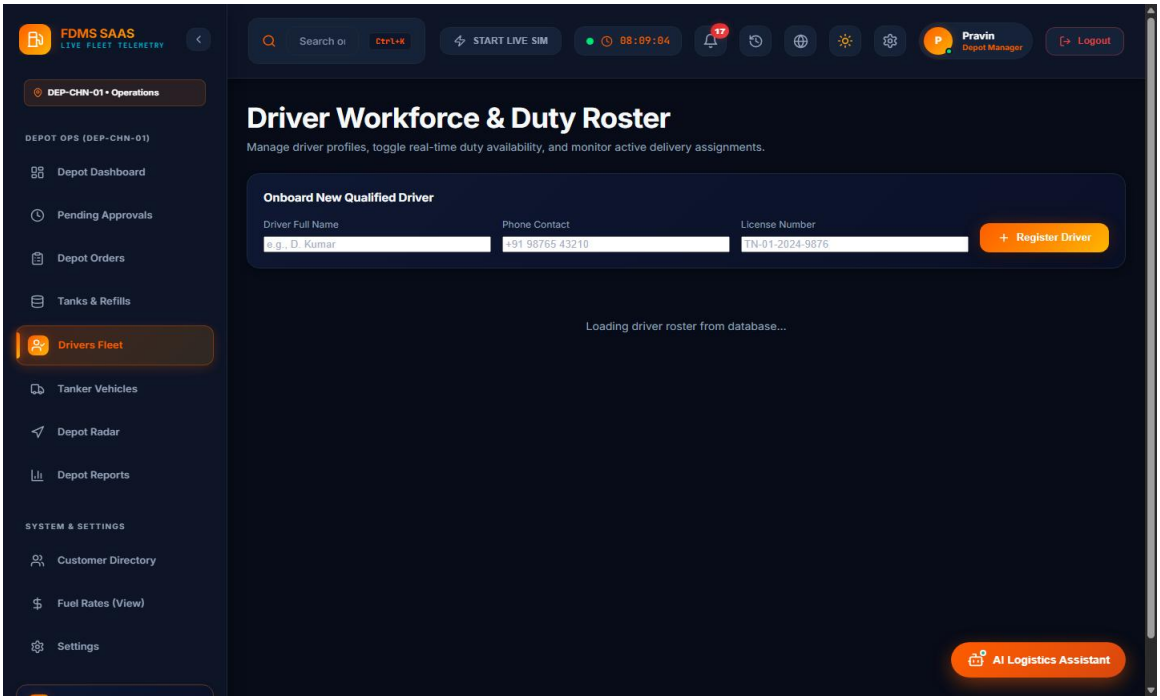


fig 8.3.6 Onboarding a new delivery driver

The driver registration modal captures new operator credentials including full name, mobile telephone number, statutory hazardous materials driving license ID, and initial terminal depot assignment before enabling dispatch login privileges.

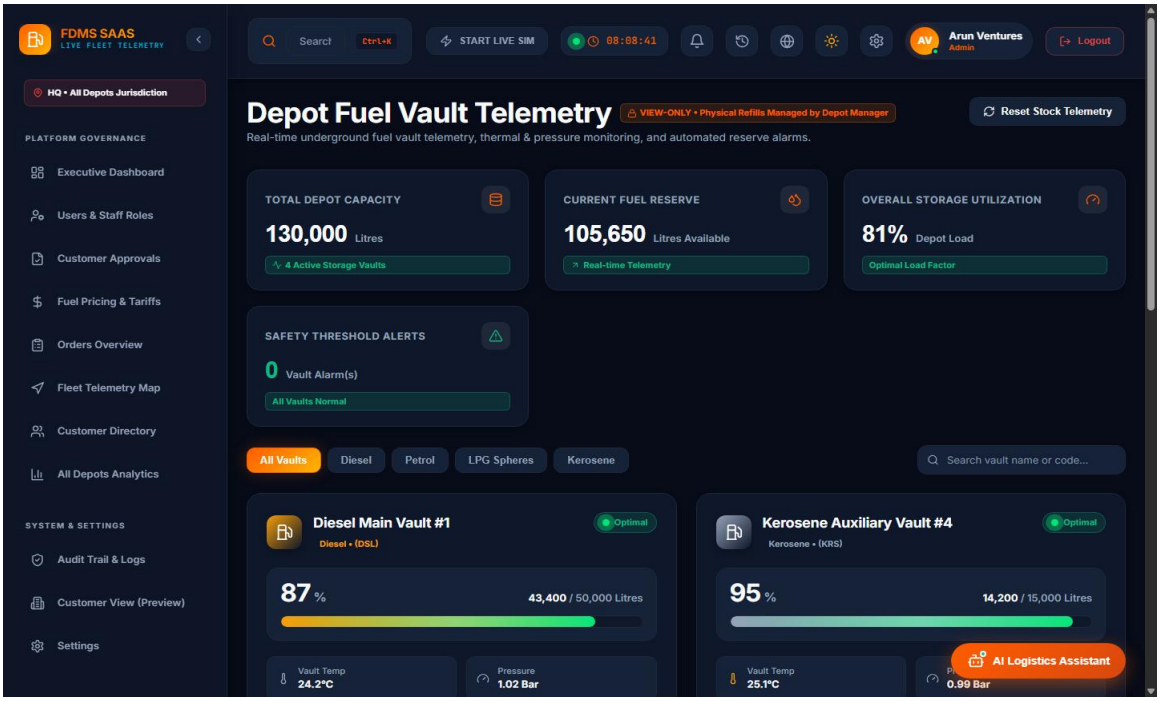


fig 8.3.8 Filtering storage tanks by fuel type

The storage vault filter isolates stationary storage tanks by designated fuel commodity code (DSL, PTL, LPG, KRS). The view displays current volumes, fill percentages, and statutory threshold alarms for immediate inventory inspection.

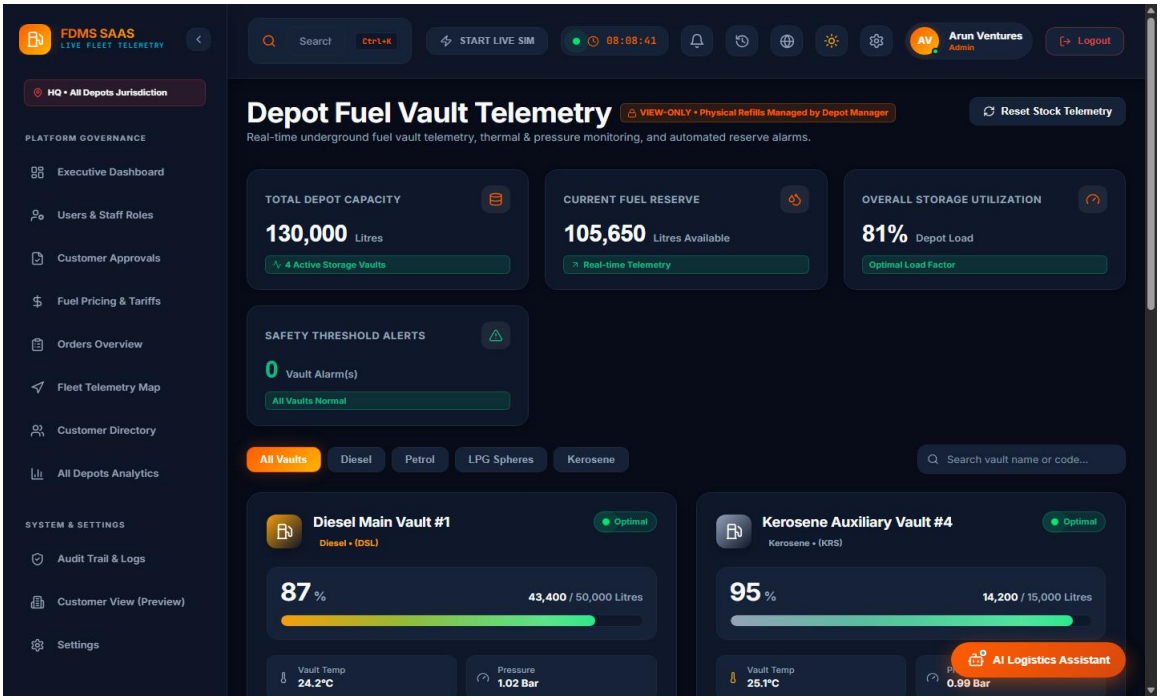


fig 8.3.9 Regional terminal storage aggregation

The multi-depot terminal storage summary aggregates physical inventory levels across Chennai, Salem, Madurai, Trichy, and Coimbatore hubs. This high-level overview enables platform administrators to monitor state-wide fuel reserves.

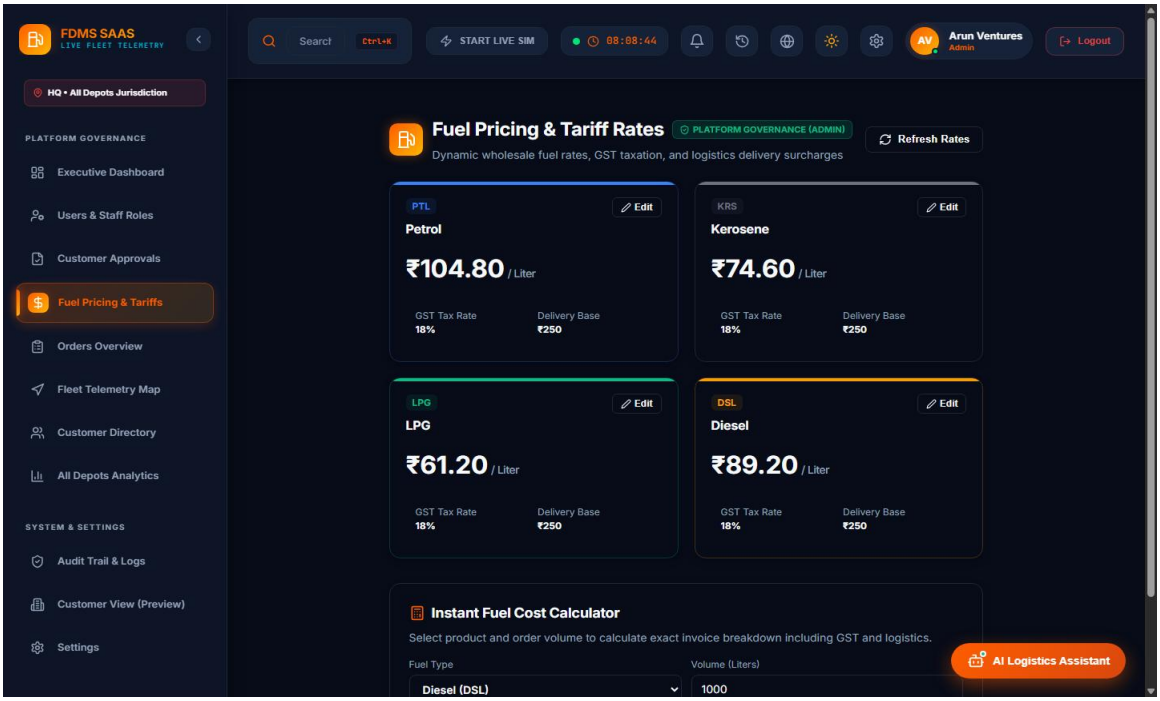


fig 8.3.10 Wholesale fuel tariff master list

The fuel tariff master schedule lists official wholesale base prices per litre for Diesel, Petrol, LPG, and Kerosene alongside applicable GST tax rates (18%) and fixed hazardous freight fees. These values govern all automated order financial computations.

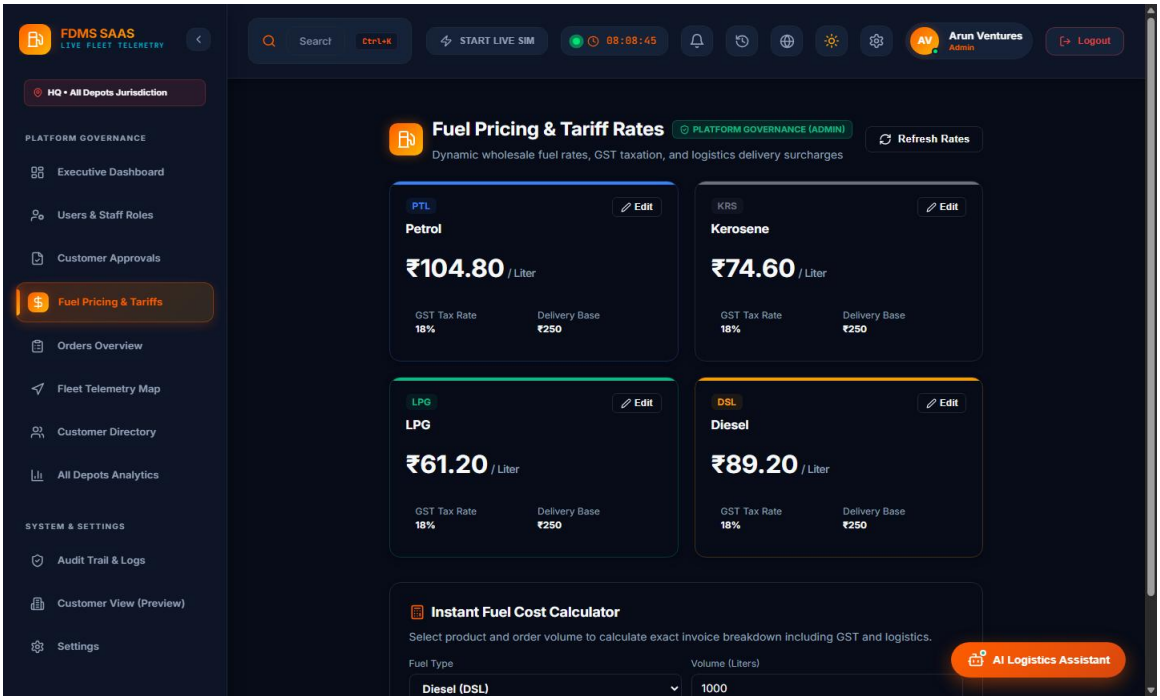


fig 8.3.11 Revising wholesale fuel tariff rate

The tariff modification dialog allows authorized Platform Administrators to revise base wholesale rates in response to daily global crude fluctuations and OMC price revisions. Updated tariffs automatically take effect on newly submitted customer purchase orders.

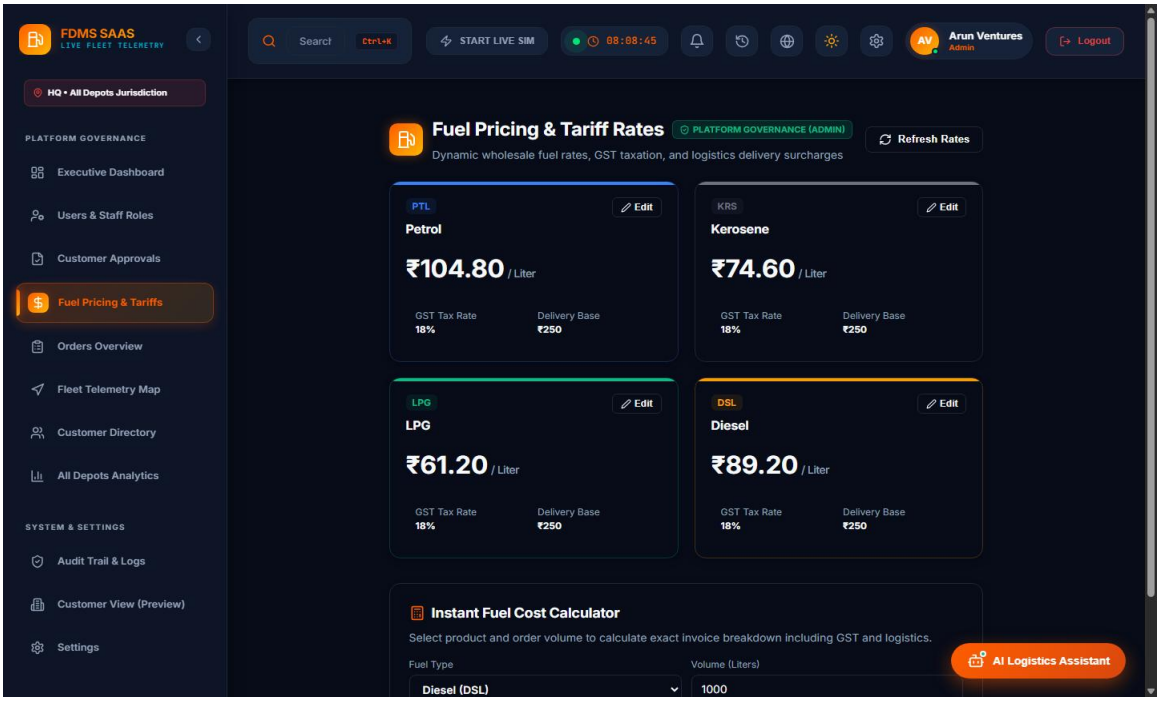


fig 8.3.12 Defining new fuel commodity tariff

The new fuel commodity modal introduces specialized fuel variants such as Bio-Diesel (BIO-D) or Aviation Turbine Fuel (JET-A1), capturing commodity codes, commercial names, base rates, and statutory tax parameters.

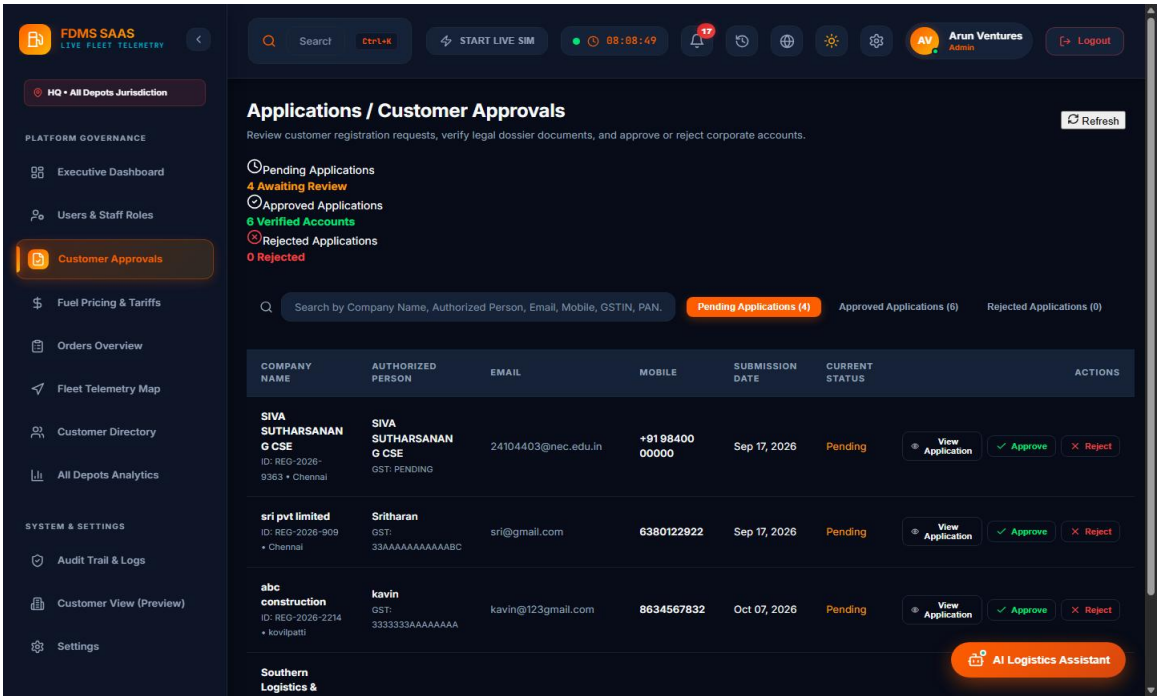


fig 8.3.13 Corporate customer KYC applications

The customer applications registry lists commercial enterprises seeking credit fuel accounts. The table summarizes registration identifiers, corporate names, verified GST numbers, Income Tax PANs, and current approval statuses.

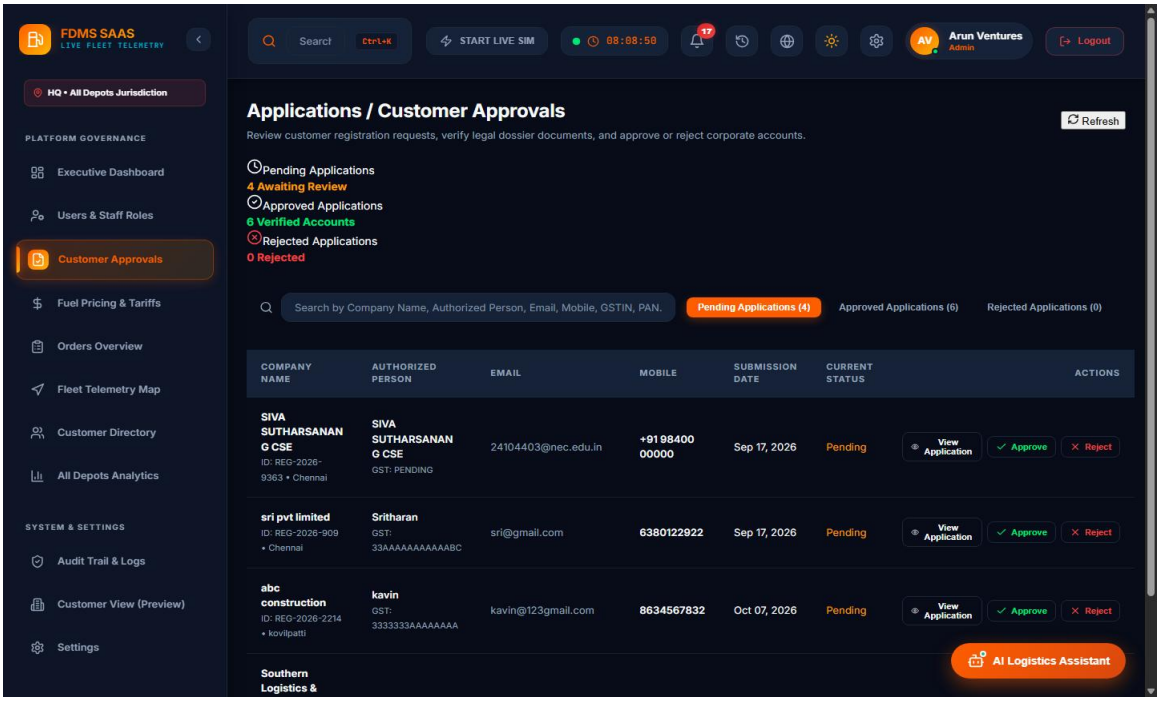


fig 8.3.14 Corporate KYC document verification

The KYC verification dialog allows platform administrators to inspect uploaded commercial certificates (GST registration, PAN cards, board resolutions) and assign approved credit limits prior to activating corporate purchasing privileges.

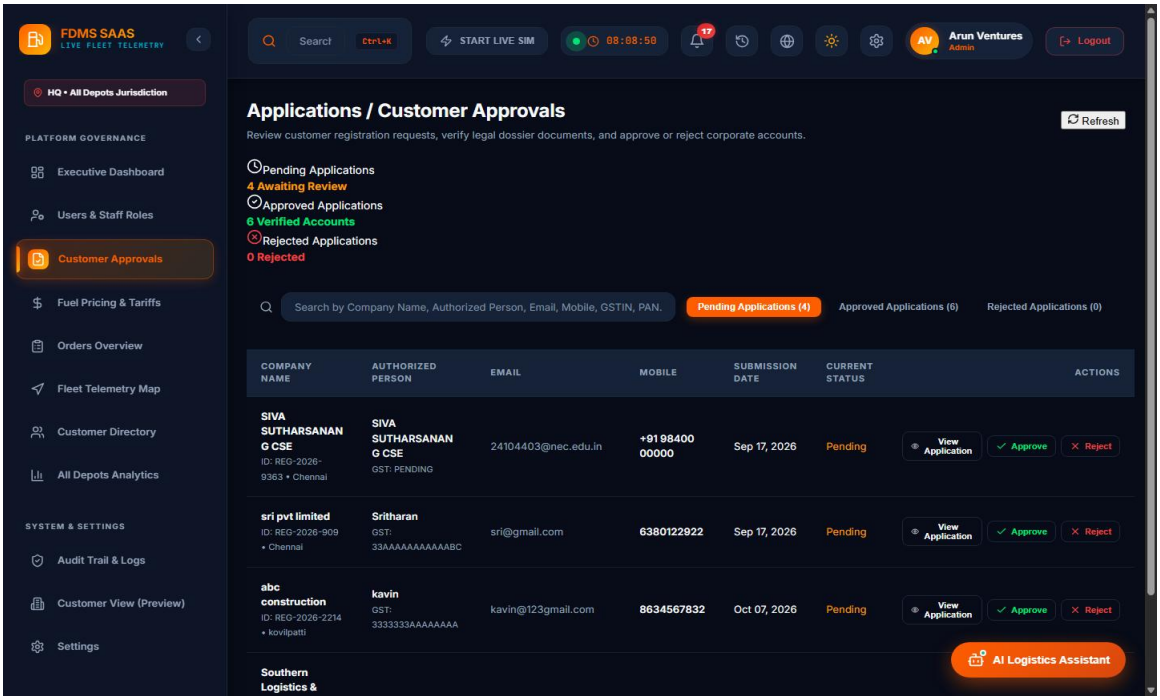


fig 8.3.15 Regional depots terminal master

The terminal registry records the 5 strategic logistics hubs across Tamil Nadu with their geographic locations, assigned terminal managers, and jurisdictional district coverages, establishing strict operational boundaries.

8.4 COMMERCIAL BULK FUEL ORDERS & CUSTOMER PORTAL

8.4.0 Purpose & Scope

Orchestrates commercial customer bulk fuel requisitions, dynamic wholesale price tariff calculation, automated tax accounting, delivery window slotting, and corporate credit limit governance. Enables commercial clients to submit digital purchase orders while enforcing terminal inventory reservations and statutory hazardous freight surcharges.

8.4.1 Functional Requirements

What the module must do:

- Commercial Order Entry: Verified corporate buyers select fuel commodity (DSL, PTL, LPG, KRS), volume in litres, delivery site address, and scheduled delivery window (e.g. 06:00-08:00, 08:00-10:00).
- Unique Sequential Numbering: Automatically generates sequential consignment identifiers conforming to the ORD-YYMMDD-XXX convention (e.g. ORD-260801-001).
- Dynamic Financial Engine: Calculates basic commodity subtotal (volume x wholesale price), statutory 18% GST (CGST 9% + SGST 9%), fixed hazardous transport delivery charge (Rs. 250), and rounded grand total.
- Credit Limit Validation: Verifies that the order value does not exceed the commercial client's available credit limit (creditLimit - creditUsed) prior to submission.
- Lifecycle State Management: Tracks order progression through defined finite states: Pending -> Approved -> Assigned -> Dispatched -> In Transit -> Delivered / Rejected / Cancelled.
- Consignment Register: Provides searchable and filterable views by order number, customer name, status, fuel type, and delivery slot with expandable detail panels.
- Order Modification & Cancellation: Permits customers to cancel unassigned orders with mandatory justification before terminal tanker dispatch.

8.4.2 Non-Functional Requirements

How well the module must do it:

- Financial precision: All subtotal, tax, surcharge, and grand total values are rounded to two decimal places, preventing floating-point arithmetic drift. Example: 500 L DSL @ Rs. 92.40 = Rs. 46,200.00 subtotal; 18% GST = Rs. 8,316.00; delivery surcharge = Rs. 250.00; Net total = Rs. 54,766.00.
- Atomicity: Order documents and customer credit allocations commit atomically; half-saved orders are prevented.
- Concurrency protection: Terminal vault stock availability is checked at order submission to avoid over-promising unallocated reserves.
- Server-side validation: Client-submitted calculations are re-computed and verified by calculateOrderPricing() in orderController prior to persistence.
- Auditability: Records creation timestamp, modifying actors, and approval timestamps on all order documents.
- Responsiveness: Order register loads in under 300 ms with lazy expansion of consignment route details.

8.4.3 Tech Stack

Concern	Choice	Why it fits this module
Order placement form	React Modal with live preview	Provides real-time tax and invoice total computation as user adjusts volume slider
State machine	OrdersContext (useReducer)	Encapsulates order status transitions in a deterministic state machine
Numbering generator	RegExp countDocuments() query	Allocates sequential ORD-YYMMDD-XXX numbers without database lock overhead
Persistence	Mongoose Order schema	Strict validation of volume minimums, delivery slots, and status enumeration

8.4.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Orders per day	~ 30 consignments	Depot operational capacity across 5 regional terminals
Orders per year	30 x 365 ~ 10,950 orders	Continuous annual distribution throughput
Storage per order	~ 2.5 KB	Order metadata, timestamps, addresses, and delivery proof
Annual storage	~ 27 MB / year	10,950 x 2.5 KB (extremely lightweight document footprint)
API calls / order	~ 4 calls	Tariff lookup, credit verification, creation, status polling

8.4.5 Low-Level Architecture

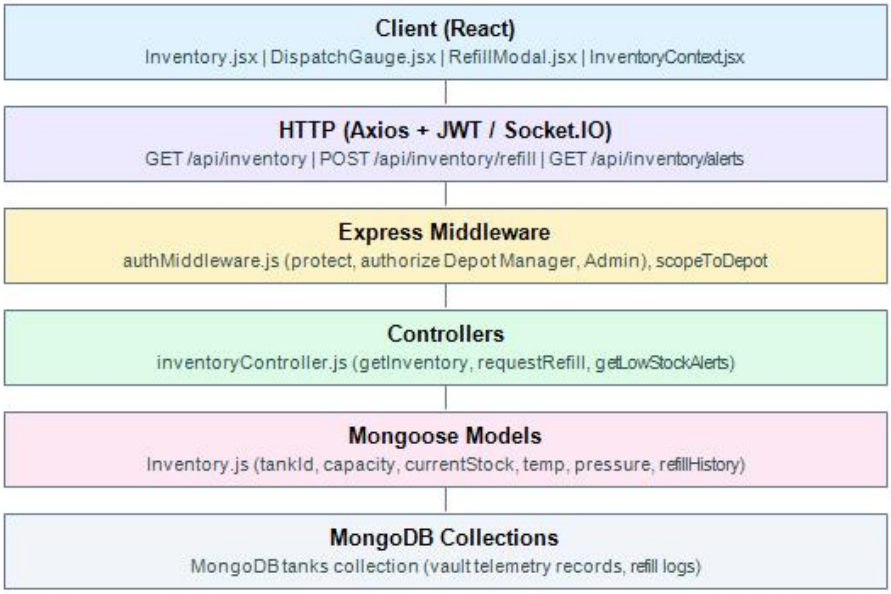


Fig 8.4.A Low-level architecture - DEPOT STORAGE VAULTS & INVENTORY TELEMETRY

Fig 8.4.A Low-level architecture - COMMERCIAL BULK FUEL ORDERS & CUSTOMER PORTAL

Request flow: * CustomerPortal.jsx or Orders.jsx opens NewOrderModal.jsx, fetching current fuel tariffs from /api/pricing. * User inputs required volume and destination; the interface immediately computes subtotal, GST (18%), and delivery surcharge. * Save sends payload via POST /api/orders; orderController re-verifies tariff rates against FuelPricing and validates customer creditLimit. * Upon successful creation, the new order receives status 'Pending' and triggers a WebSocket notification to the Depot Manager console. * Depot Managers review the consignment in PendingRequests.jsx and either approve or reject with reason.

8.4.6 API Design

Method	Endpoint	Purpose / contract
GET	/api/orders	Lists orders with status, depot, and customer filters; returns paginated array
POST	/api/orders	Customer / Admin. Creates new fuel order; server recomputes financial totals
GET	/api/orders/:id	Retrieves complete order record with vehicle, driver, and delivery timeline
PUT	/api/orders/:id	Depot Manager only. Updates consignment details prior to dispatch assignment
DELETE	/api/orders/:id	Admin only. Deletes an order record from active operations
PATCH	/api/orders/:id/status	Updates order workflow status (Driver delivery or

Method	Endpoint	Purpose / contract
		Customer cancellation)

8.4.7 DB Design

Table / Collection	Keys & indexes	Key columns
O		
P		
o		

* Unique index on orderNumber guarantees tamper-proof consignment sequencing. * Compound indexes on (depot, status) ensure sub-second retrieval for terminal dispatch queues.

8.4.8 Screenshots

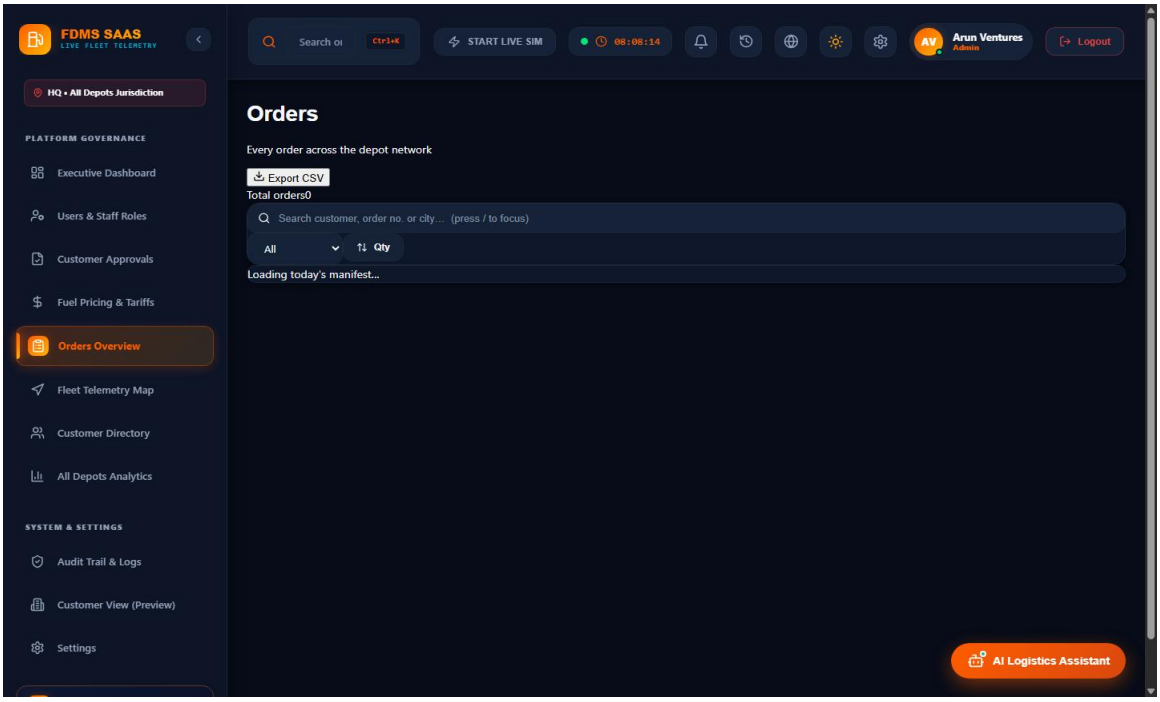


fig 8.4.1 Commercial bulk fuel orders register

The bulk orders register presents all customer consignments with their unique order numbers, corporate customer names, fuel commodity types, volumes, delivery slots, workflow status pills, and net invoice amounts. Search and filter controls assist operators in monitoring fulfillment pipelines.

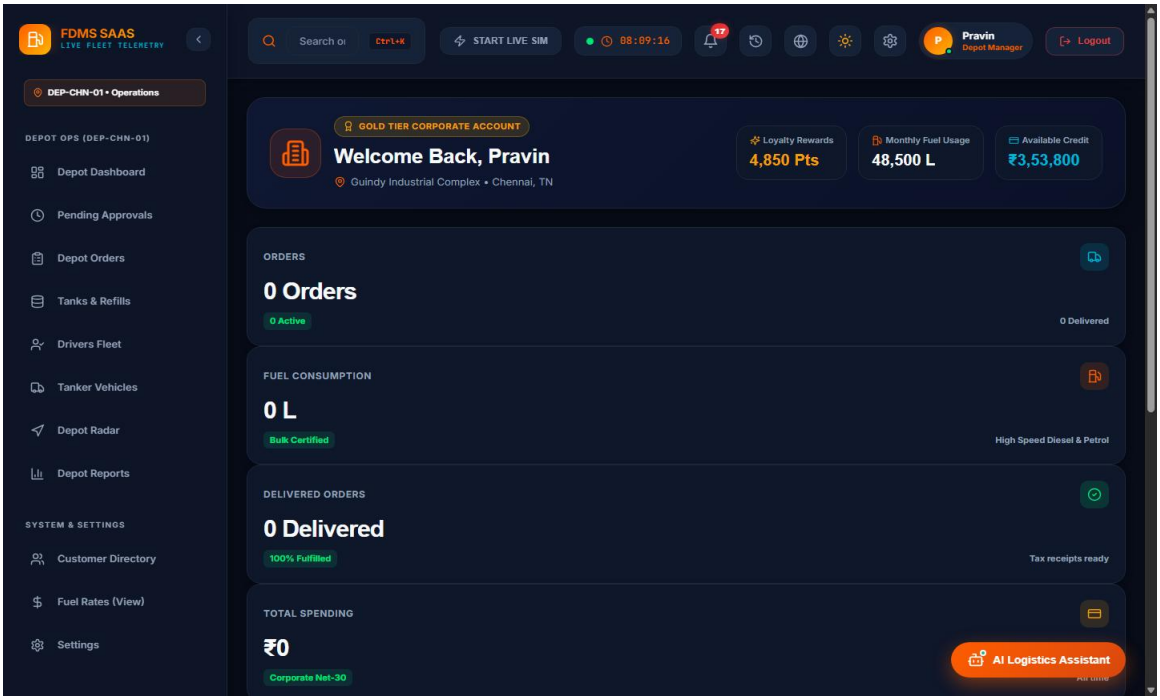


fig 8.4.2 Creating a commercial bulk fuel purchase order

The order creation modal allows commercial customers or dispatch clerks to configure a new consignment. The user selects the customer account, fuel commodity (HSD Diesel), consignment volume, and scheduled delivery time slot before submitting the requisition.

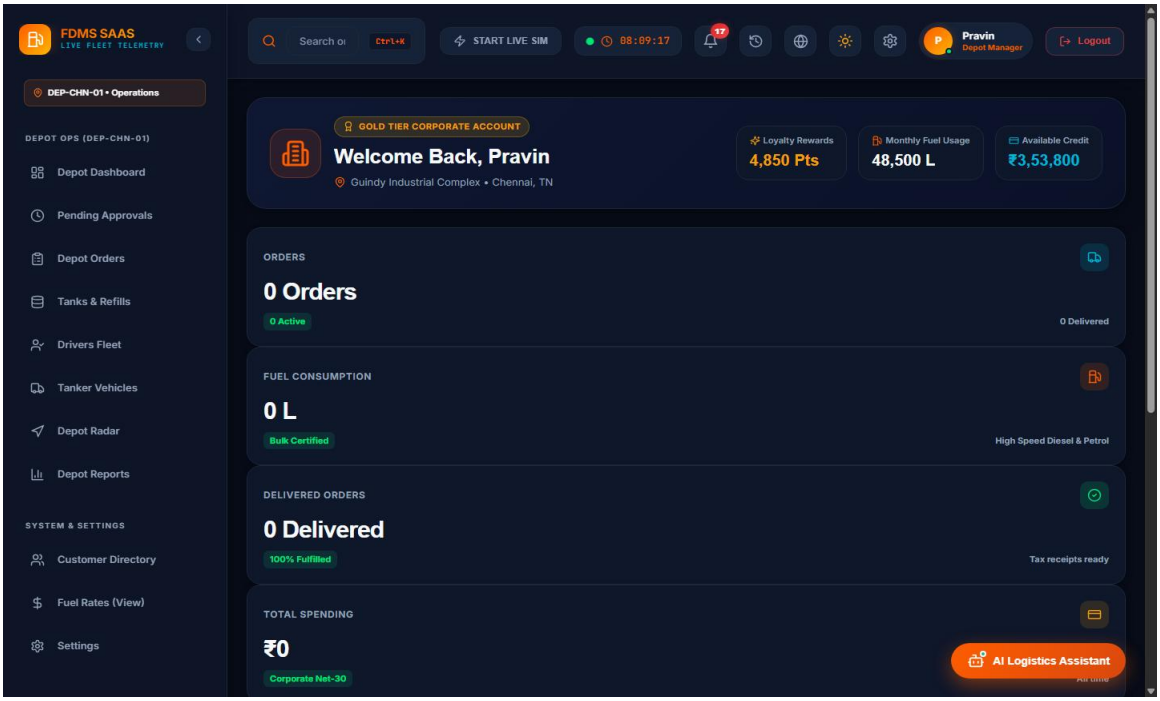


fig 8.4.3 Fuel order item tariff calculation

The dynamic pricing engine computes the exact financial breakdown in real time. The modal illustrates 500 Litres of Diesel at Rs. 92.40 per litre yielding a subtotal of Rs. 46,200.00, applicable 18% GST of Rs. 8,316.00, hazardous delivery freight of Rs. 250.00, and computed grand total of Rs. 54,766.00.

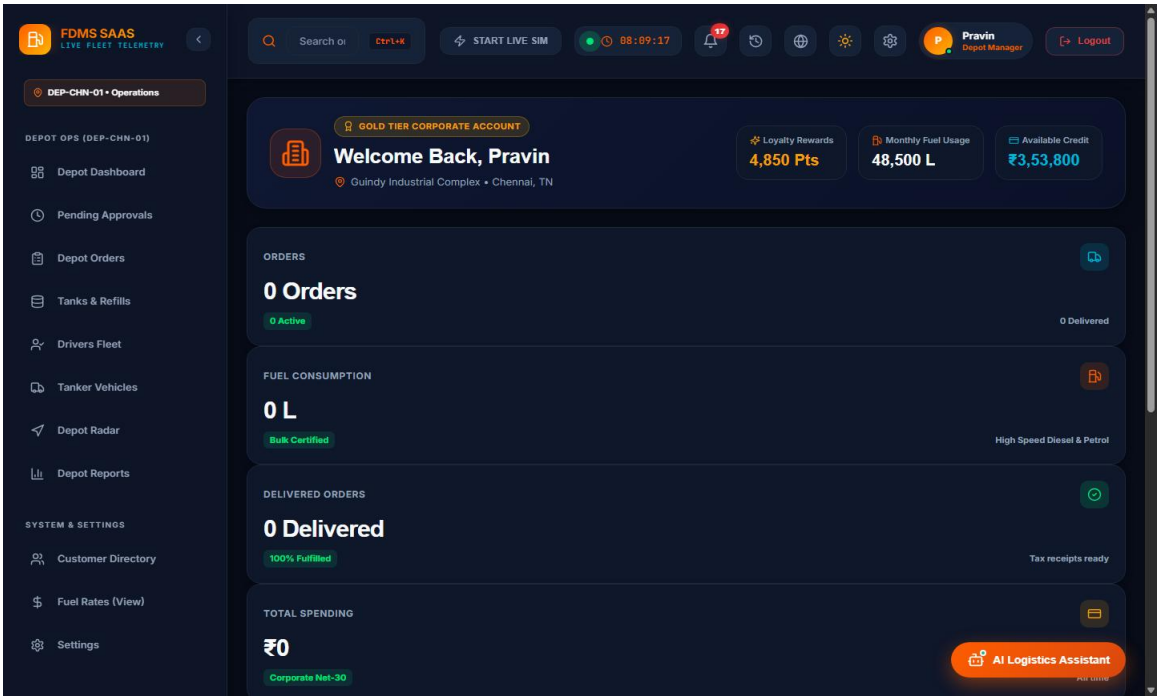


fig 8.4.4 Commercial order financial summary & tax breakdown

The order financial summary presents an itemized breakdown of commodity charges, taxes, and freight surcharges prior to final submission. Reviewing this breakdown guarantees transparency and prevents commercial billing disputes.

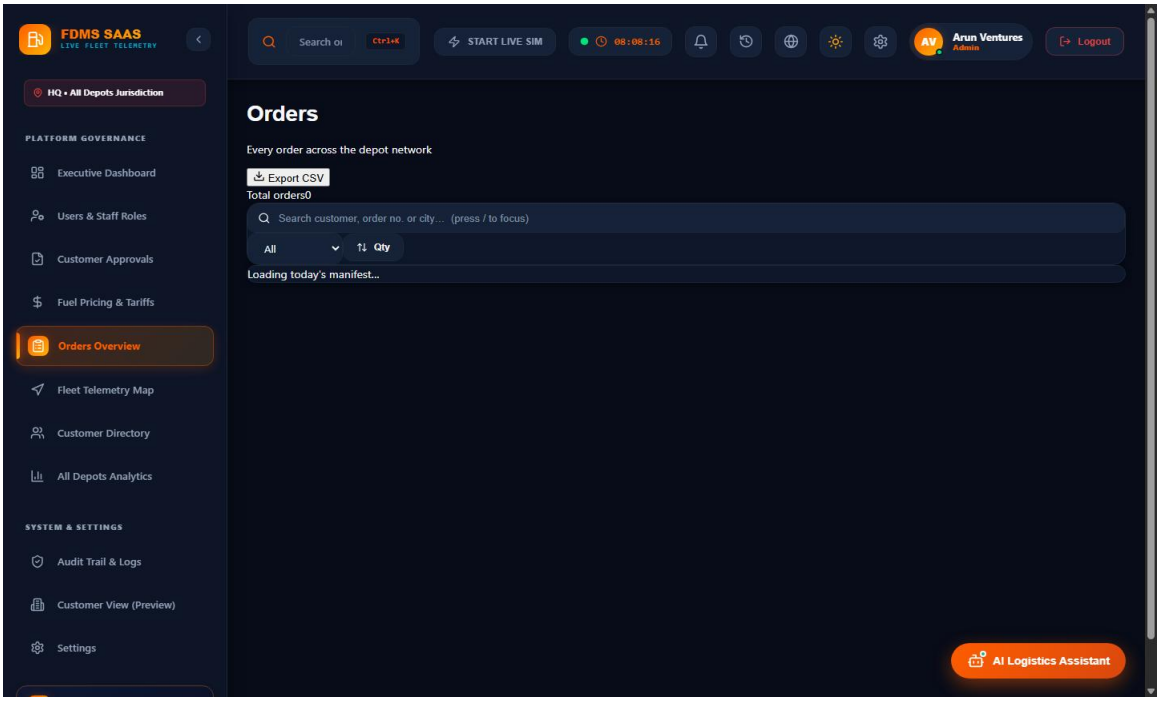


fig 8.4.5 Expanded order details & delivery site coordinates

Expanding an individual consignment reveals granular dispatch details including recipient site address, GPS delivery coordinates, assigned tanker plate number, driver contact information, and delivery timestamp logs.

8.5 SMART FUEL DISPATCH & ORDER FULFILLMENT ENGINE

8.5.0 Purpose & Scope

Orchestrates order authorization, intelligent heuristic tanker-driver pairing, physical terminal stock reservation, delivery queue dispatching, discharge telemetry verification, and automated computerized GST tax invoice generation. Connects customer bulk requisitions with active terminal transport resources.

8.5.1 Functional Requirements

What the module must do:

- Order Approval Workflow: Depot Managers review incoming Pending orders, verify customer credit health, and approve or reject with recorded justification.
- Terminal Stock Reservation: Authorizing an order automatically reserves the required fuel volume in the corresponding depot storage vault (Inventory.reserved).
- Heuristic Dispatch Suggestion: Evaluates available fleet tankers based on dedicated fuel cargo compatibility (DSL, PTL, LPG, KRS), payload capacity, on-duty driver state, and route distance via getSmartDispatchSuggestion.
- Consignment Assignment: Assigns calibrated tanker plate and certified on-duty driver, advancing status from Approved to Assigned.
- Delivery Queue Manifest: Generates the active terminal dispatch queue for gate outwards and driver field delivery scheduling.
- Discharge Verification & Signature: Captures driver flow meter discharge readings, arrival timestamps, and digital customer signatures upon delivery completion.
- Automated GST Invoice Generation: Produces computerized GST tax invoices with QR verification codes, tax breakdowns (CGST/SGST/IGST), and formal commercial terms.

8.5.2 Non-Functional Requirements

How well the module must do it:

- Strict jurisdictional security: Enforces that only Depot Managers assigned to the target terminal can approve orders and dispatch tankers; Admins receive HTTP 403 on physical dispatch.
- Atomic state progression: Status transitions (Approved -> Assigned -> Dispatched -> Delivered) execute atomically with vehicle state locks.
- No over-allocation: Stock availability and vehicle capacity are verified synchronously before assignment is finalized.
- Auditability: Every approval, assignment, status advancement, and rejection is recorded in the central ActivityLog with user credentials.
- Invoice generation speed: Computerized GST tax invoices render and download as print-ready documents in under 500 ms.

8.5.3 Tech Stack

Concern	Choice	Why it fits this module

Concern	Choice	Why it fits this module
Approval & dispatch UI	PendingRequests.jsx + Modals	Dedicated interface for rapid review, algorithmic pairing, and dispatch scheduling
Heuristic matching engine	orderController dispatch algorithm	Evaluates multi-variable vehicle cargo compatibility, volume, and driver duty state
Invoice presentation	InvoiceModal.jsx	Professional printable tax invoice modal adhering to Indian GST statutory requirements
Real-time notifications	Socket.IO event broadcasting	Immediately notifies customer and driver upon consignment assignment and dispatch

8.5.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Consignments dispatched / day	~ 25 consignments	Depot daily operational throughput
Dispatch operations / year	~ 9,125 dispatches	25 x 365 operational days
Invoice storage size	~ 3.5 KB / document	Structured JSON manifest and PDF metadata
Heuristic match latency	P95 < 45 ms	In-memory evaluation across local depot fleet

8.5.5 Low-Level Architecture

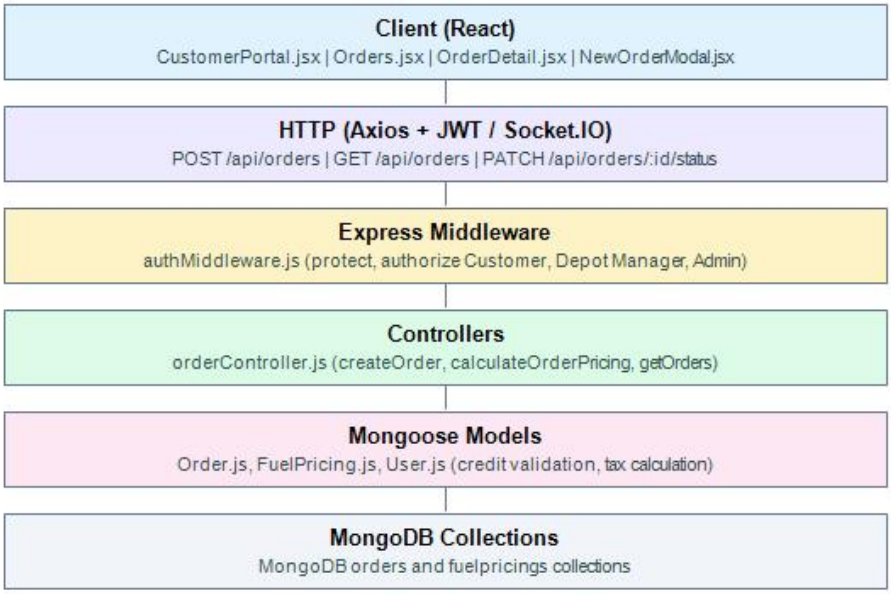


Fig 8.5.A Low-level architecture - COMMERCIAL FUEL ORDERS & CUSTOMER PORTAL

Fig 8.5.A Low-level architecture - SMART FUEL DISPATCH & ORDER FULFILLMENT ENGINE

Request flow: * Depot Manager accesses PendingRequests.jsx, viewing unassigned customer requisitions. * Approving an order issues PATCH /api/orders/:id/approve; orderController locks storage stock in Inventory and sets status 'Approved'. * Clicking 'Dispatch' calls /api/orders/:id/smart-dispatch; the heuristic algorithm returns candidate tankers filtered by cargo compatibility and duty state. * Submitting the assignment modal invokes PATCH /api/orders/:id/assign; the vehicle status transitions to 'Assigned', the driver is allocated, and a WebSocket event is broadcast. * Upon site delivery, DriverPortal submits flow meter proof and digital signature via PATCH /api/orders/:id/status; status updates to 'Delivered' and InvoiceModal generates the tax invoice.

8.5.6 API Design

Method	Endpoint	Purpose / contract
GET	/api/orders/pending	Lists unapproved orders awaiting depot manager authorization
PATCH	/api/orders/:id/approve	Depot Manager / Admin. Authorizes order and reserves depot tank stock
PATCH	/api/orders/:id/reject	Depot Manager / Admin. Rejects order with mandatory rejectionReason
GET	/api/orders/:id/smart-dispatch	Returns heuristic recommendations for optimal tanker and on-duty driver
PATCH	/api/orders/:id/assign	Depot Manager only. Binds selected tanker and driver to consignment

Method	Endpoint	Purpose / contract
GET	/api/orders/delivery-queue	Returns active dispatch manifest for gate outward scheduling
GET	/api/orders/:id/invoice	Generates computerized GST tax invoice data with breakdown and QR code

8.5.7 DB Design

Table / Collection	Keys & indexes	Key columns
Orders	PK _id INDEX status, depot INDEX driver, vehicle	orderNumber, status ('Approved','Assigned'...), vehicle, driver, deliveryProof, customerSignature, approvedAt, assignedAt, reachedAt
Tanks	PK tankId INDEX depot, fuelType	capacity, currentStock, reserved (fuel allocated to active dispatches)

8.5.8 Screenshots

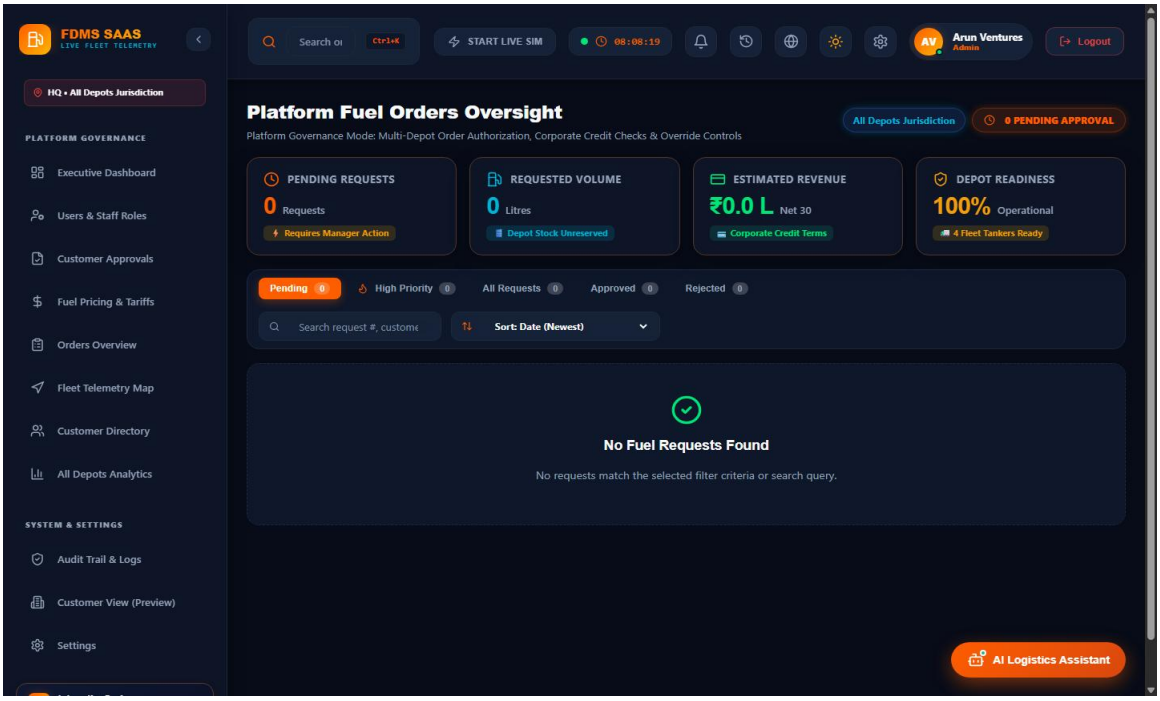


fig 8.5.1 Pending fuel order approvals queue

The pending approvals queue lists newly placed commercial fuel orders requiring terminal administrative review. Operators can review requested volume, fuel type, requested time window, and initiate authorization.

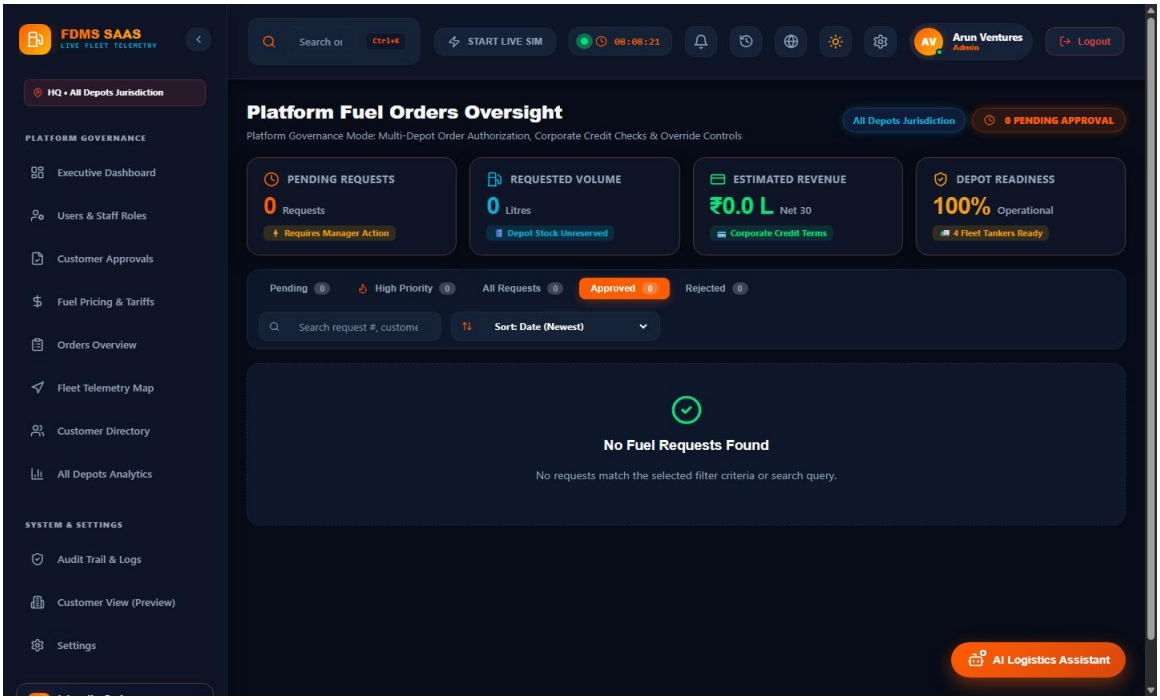


fig 8.5.2 Depot manager order authorization

The order authorization modal verifies corporate client credit limits and terminal storage tank stock availability prior to approving the consignment and reserving the inventory.

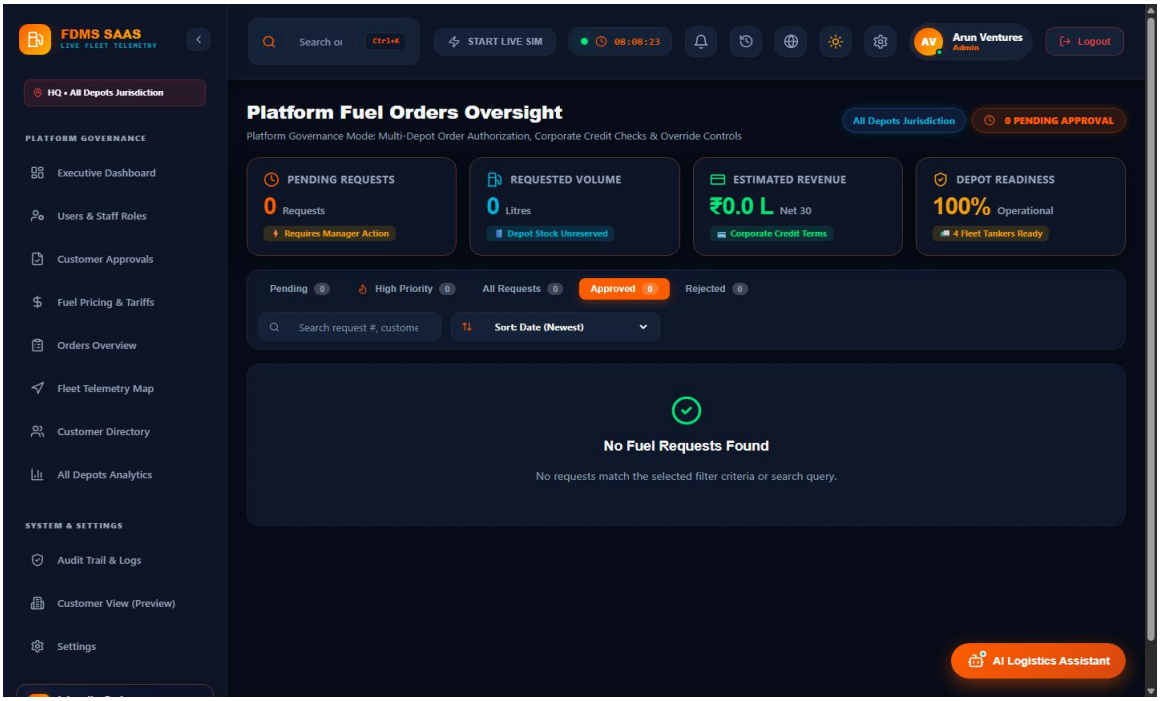


fig 8.5.3 Smart heuristic dispatch engine suggestion

The heuristic dispatch suggestion dialog analyzes available fleet tankers and on-duty drivers, recommending the optimal vehicle based on fuel compartment compatibility, payload capacity, and geographic proximity.

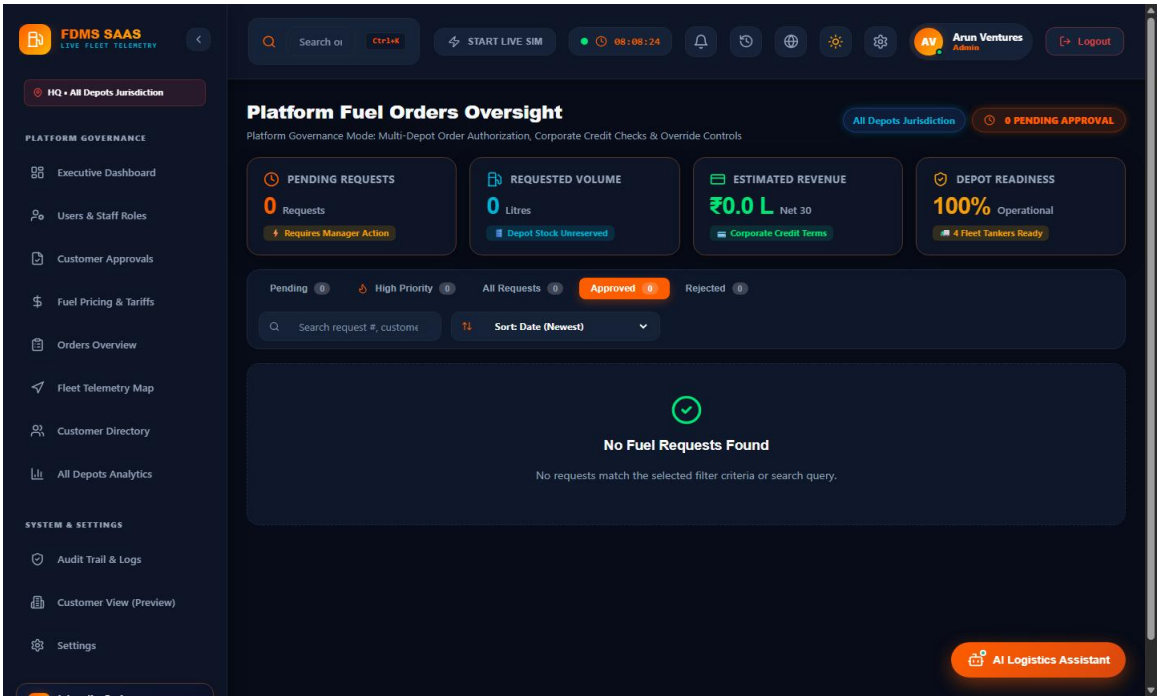


fig 8.5.4 Assigning delivery tanker & driver to order

The dispatch assignment modal binds the recommended tanker vehicle and certified on-duty driver to the order, advancing the consignment status to Assigned and locking the transport asset.

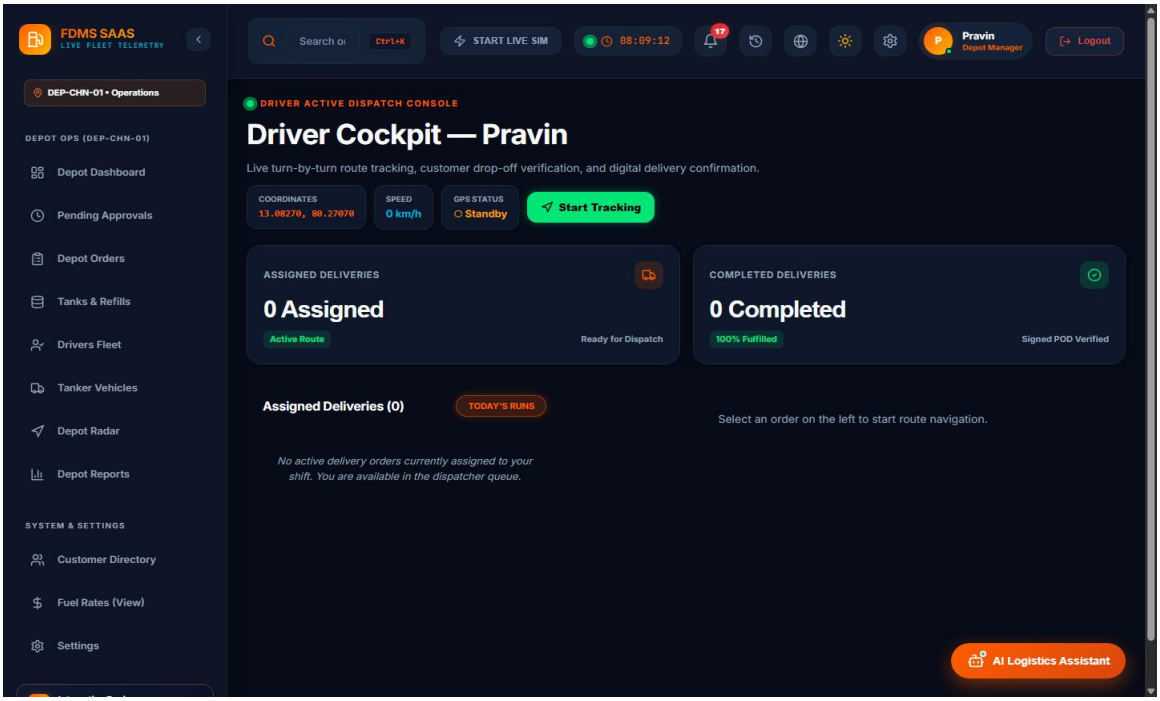


fig 8.5.5 Active terminal delivery queue & manifest

The terminal delivery queue details active consignments departing the terminal gate, tracking departure timestamps, destination addresses, and assigned tanker drivers.

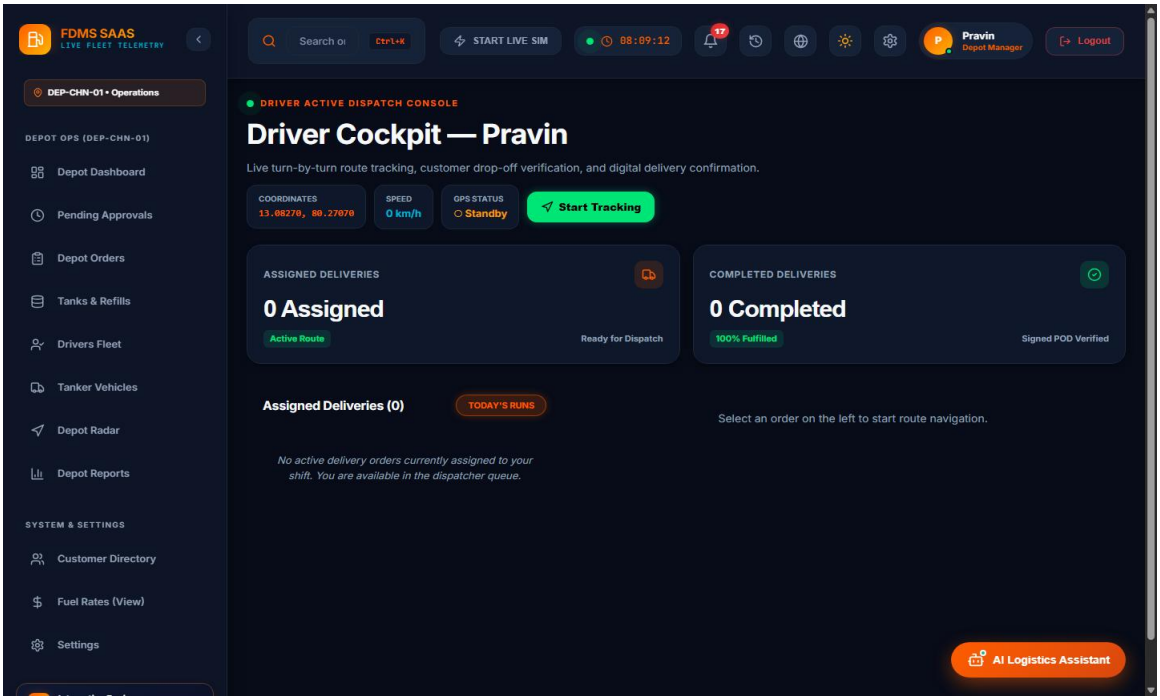


fig 8.5.6 Proof of discharge & consignment verification

The proof of discharge verification panel records initial and final flow meter readings from the tanker, confirming accurate fuel delivery before closing the consignment.

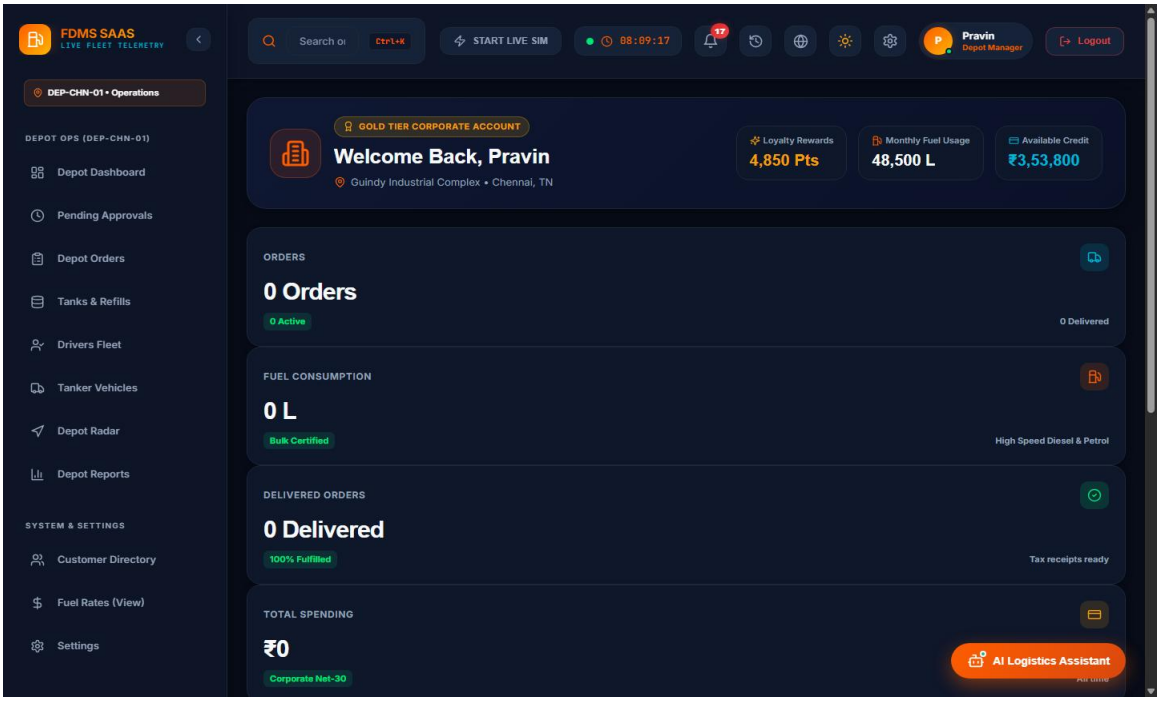


fig 8.5.7 Computerized GST tax invoice generation

The computerized tax invoice modal generates an official GST-compliant tax invoice detailing supplier GSTIN, recipient information, commodity HSN codes, tax breakdown (CGST+SGST), and net payable amount.

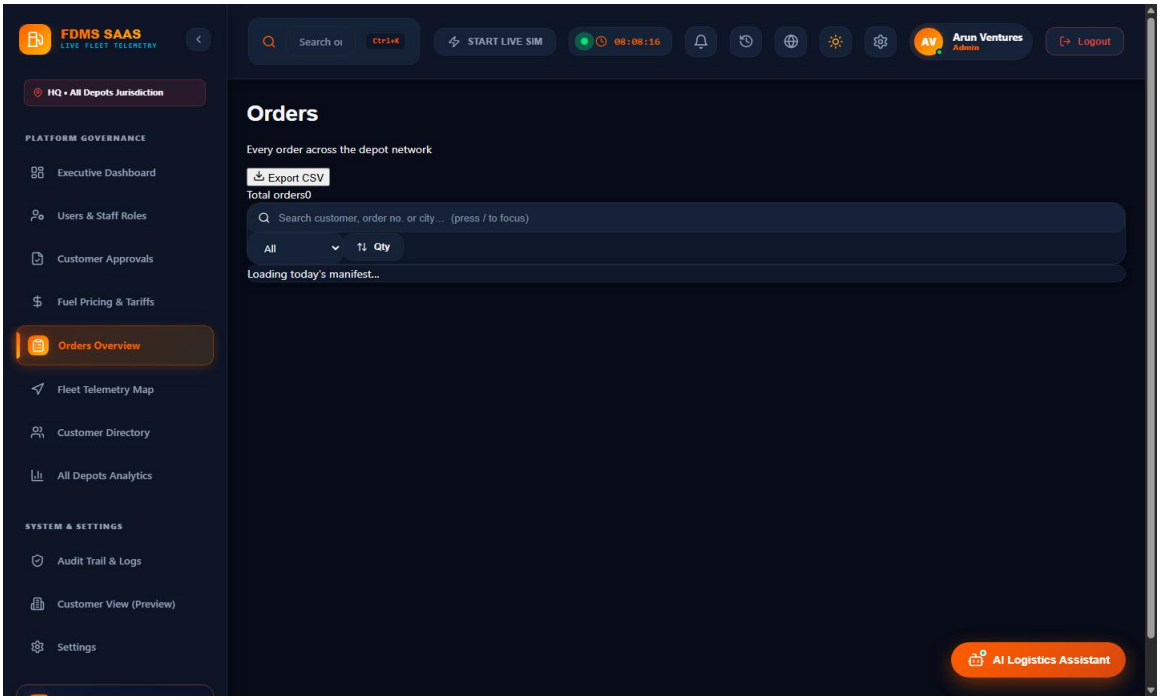


fig 8.5.8 Expanded order fulfilment & lifecycle trail

The expanded order audit trail records chronological milestones across the consignment lifecycle from initial order creation to final customer signature and invoicing.

8.6 LIVE GPS FLEET TELEMETRY & DRIVER PORTAL

8.6.0 Purpose & Scope

Regulates real-time geographic vehicle positioning, sensor telemetry streaming, and driver field operations. Combines high-frequency GPS coordinate broadcasts over WebSockets with interactive Leaflet GIS mapping, route breadcrumb history, vehicle health metrics (tire pressure, engine temperature), and digital proof-of-discharge signature capture.

8.6.1 Functional Requirements

What the module must do:

- Driver Field Portal: Provides a dedicated mobile-responsive interface (DriverPortal.jsx) displaying active assigned consignments, route directions, and customer contact details.
- Live GPS Telemetry Broadcast: Broadcasts vehicle latitude, longitude, instantaneous speed, and compass heading at regular 10-second intervals via /api/gps/update.
- Real-Time GIS Mapping: Renders an interactive Leaflet map (FleetMap.jsx) displaying regional depots, moving tanker markers, destination sites, and dynamic route polylines.
- Vehicle Telemetry Sensor Inspection: Captures diagnostic telemetry including tanker fuel levels, tire pressure (PSI), engine coolant temperature (Celsius), and transit speed.
- Breadcrumb Location History: Records historical route coordinates in MongoDB (DriverLocations and Vehicle.locationHistory) for route auditing and post-trip analysis.
- Proof of Discharge & Digital Signature: Captures customer signatory name, mobile phone verification, and cryptographic touch/mouse signature upon fuel transfer.

8.6.2 Non-Functional Requirements

How well the module must do it:

- Low latency streaming: GPS location broadcasts are distributed to connected dispatcher dashboards via Socket.IO within 100 ms of receipt.
- Geospatial query optimization: Employs MongoDB 2dsphere indexes for rapid distance and proximity calculations without CPU exhaustion.
- History boundary: Caps in-document vehicle locationHistory breadcrumbs at 1,000 points to prevent document bloat beyond MongoDB's 16 MB limit.
- Fault tolerance: Caches offline telemetry on mobile clients and synchronizes automatically upon network reconnection.
- Statutory compliance: Retains tamper-proof GPS records in DriverLocations for statutory PESO hazardous consignment auditing.

8.6.3 Tech Stack

Concern	Choice	Why it fits this module
GIS map visualization	Leaflet.js + OpenStreetMap	Lightweight, responsive GIS map rendering with custom tanker icons and polylines

Concern	Choice	Why it fits this module
Real-time telemetry transport	Socket.IO WebSockets	Event-driven, low-overhead bidirectional socket streaming with auto-reconnection
Mobile driver portal	DriverPortal.jsx (React)	Touch-friendly field console optimized for mobile screens and driver field workflows
Geospatial storage	MongoDB 2dsphere index	Enables native GeoJSON Point queries and distance computations

8.6.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Active delivery tankers	~ 20 tankers in transit	Peak daytime delivery operations
Telemetry broadcast interval	10 seconds / vehicle	Balances GPS accuracy with cellular bandwidth
Telemetry pings / hour	7,200 telemetry points	20 tankers x 360 pings/hour
Telemetry storage / day	~ 15 MB / day	72,000 daily points x ~ 200 bytes per record
Map render performance	60 FPS rendering	Hardware-accelerated Leaflet canvas and SVG markers

8.6.5 Low-Level Architecture

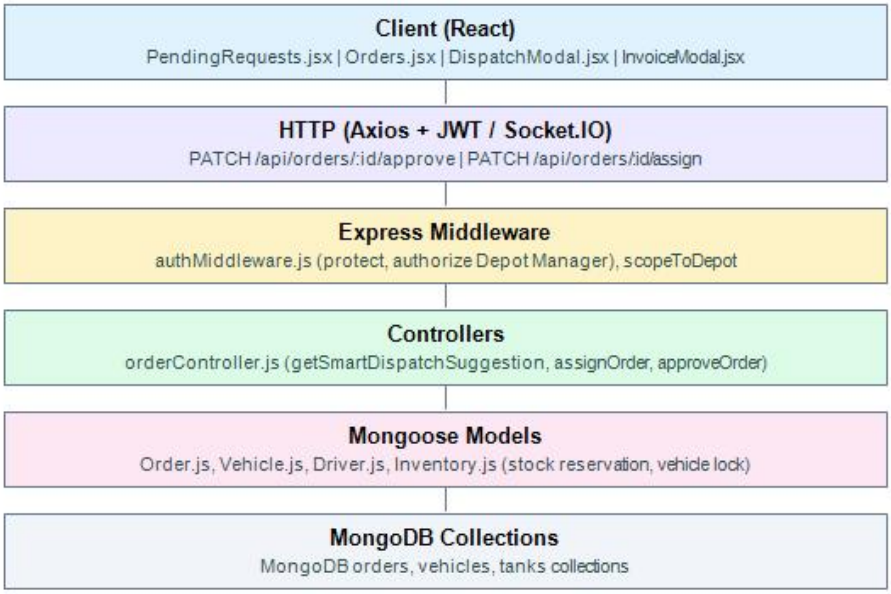


Fig 8.6.A Low-level architecture - SMART FUEL DISPATCH ENGINE

Fig 8.6.A Low-level architecture - LIVE GPS FLEET TELEMETRY & DRIVER PORTAL

Request flow: * DriverPortal.jsx reads the device's HTML5 Geolocation API, extracting latitude, longitude, and speed. * The portal posts updates to /api/gps/update; gpsController validates numerical ranges (-90 to 90, -180 to 180). * The controller records the telemetry point in DriverLocations, updates the Vehicle document, and appends to locationHistory. * Socket.IO broadcasts the update to all active dispatcher browsers listening on the 'vehicle:location' channel. * FleetMap.jsx updates the tanker marker position smoothly, draws updated route polylines, and recalculates ETA.

8.6.6 API Design

Method	Endpoint	Purpose / contract
POST	/api/gps/update	Driver / Device. Broadcasts live vehicle GPS coordinates, speed, and heading
GET	/api/gps/telemetry	Protected. Retrieves current live positions of all active fleet tankers
GET	/api/gps/history/:vehicleId	Returns historical breadcrumb coordinates for route playback
POST	/api/orders/:id/discharge	Driver only. Submits final flow meter discharge proof and signature

8.6.7 DB Design

Table / Collection	Keys & indexes	Key columns

Table / Collection	Keys & indexes	Key columns
DriverLocations	PK _id INDEX vehicleId, timestamp INDEX driverId, timestamp	vehicleId, driverId, driverName, latitude, longitude, speed, heading, accuracy, timestamp
Vehicles	PK vehicleNumber 2DSPHERE location	latitude, longitude, speed, location {type:'Point',coordinates:[lng,lat]}, locationHistory

8.6.8 Screenshots

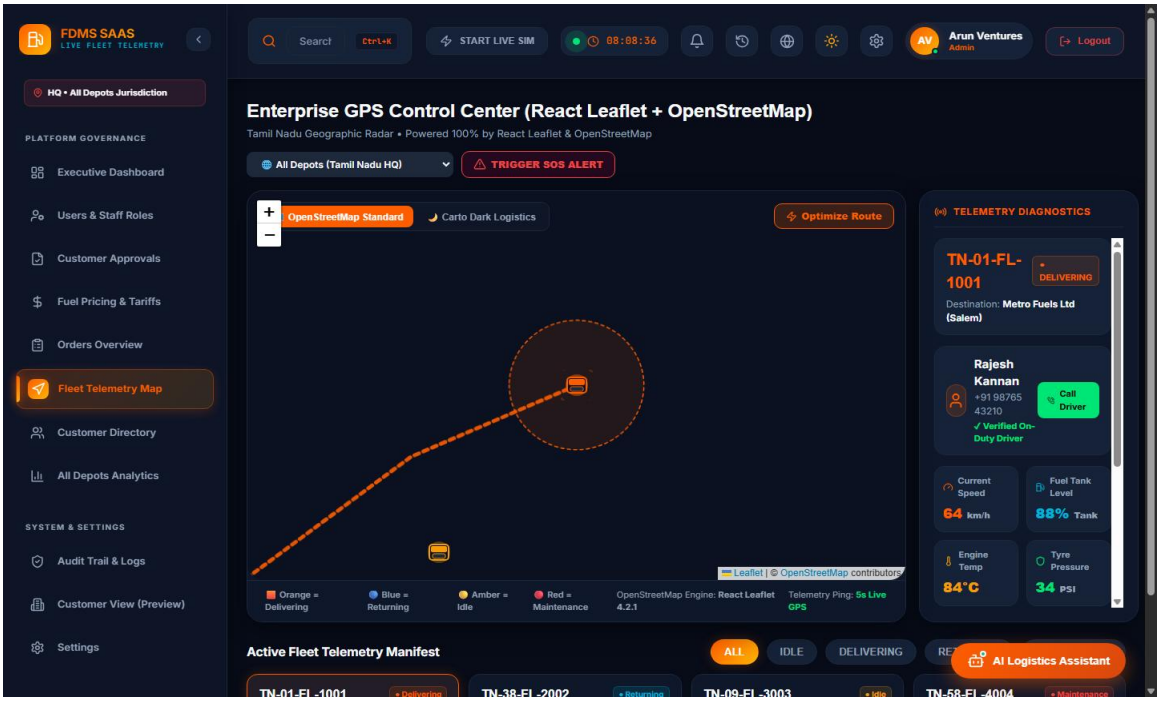


fig 8.6.1 Real-time interactive fleet telemetry map

The interactive GIS fleet map renders real-time tanker positions across Tamil Nadu highways. Clicking an individual tanker displays a telemetry card showing instantaneous speed, driver identity, dedicated cargo volume, and estimated arrival time.

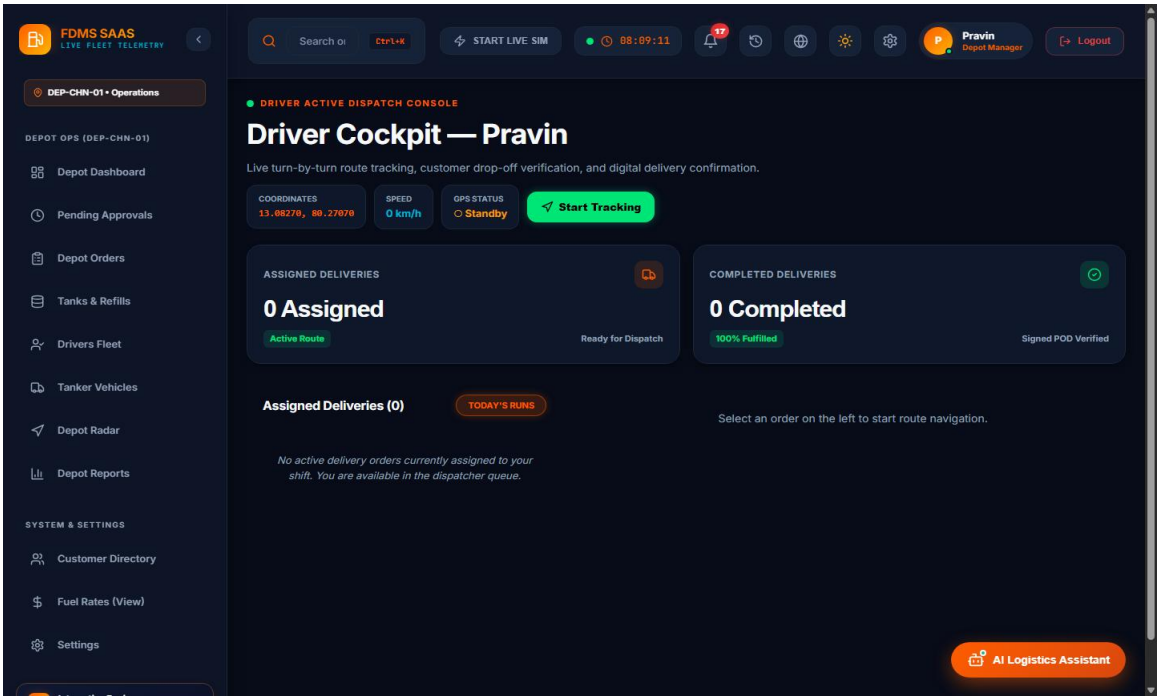


fig 8.6.2 Driver field mobile dispatch console

The driver mobile portal presents a streamlined dispatch view for tanker operators, showing current consignment details, site delivery address, real-time GPS broadcast status, and flow meter discharge entry controls.

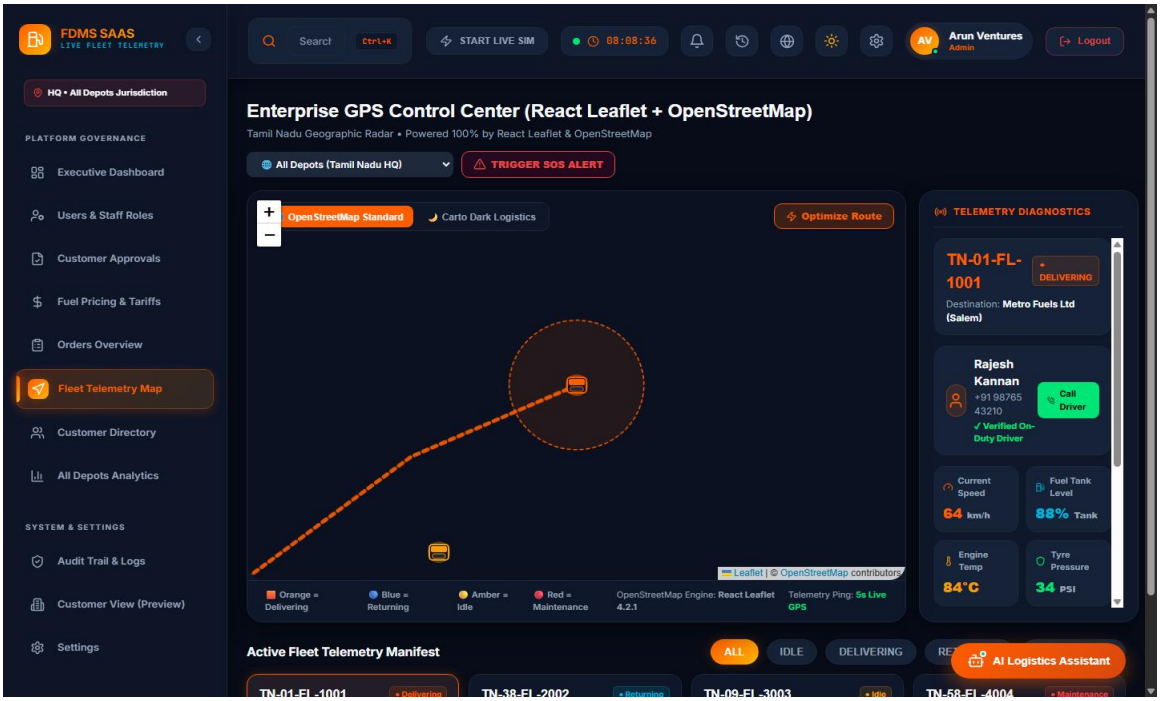


fig 8.6.3 Driver live speed & heading broadcast

The telemetry broadcast dialog confirms active GPS transmission, reporting latitude, longitude, speed (54 km/h), compass heading, tire pressure (108 PSI), and engine temperature readings to the central depot server.

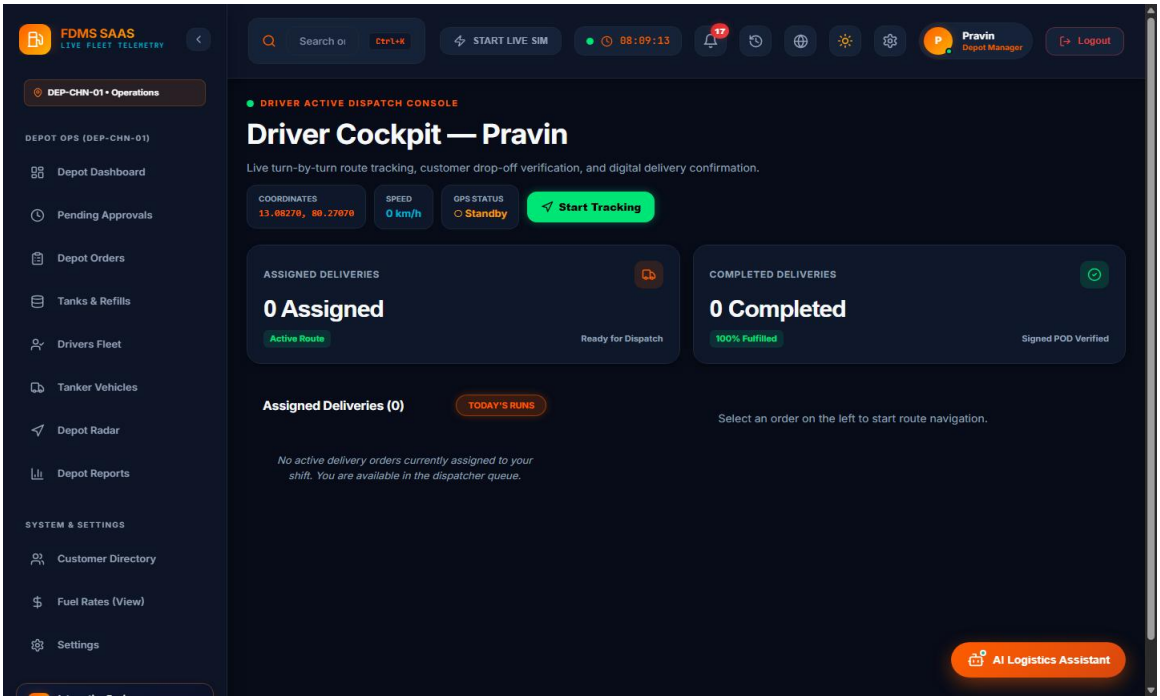


fig 8.6.4 Customer digital signature & proof of delivery

The digital proof of delivery panel captures the recipient's authorized signature, contact verification, and delivery completion timestamp, establishing legal proof of cargo discharge.

8.7 DEPOT STORAGE VAULTS & INVENTORY TELEMETRY

8.7.0 Purpose & Scope

Regulates physical terminal fuel storage, tank vault sensor telemetry, statutory minimum reorder thresholds, automated low-stock warnings, and bulk stock replenishment from Oil Marketing Companies (OMCs like IOCL, BPCL, HPCL). Guarantees physical inventory integrity and prevents terminal stockouts.

8.7.1 Functional Requirements

What the module must do:

- Vault Telemetry Monitoring: Monitors stationary storage tanks for Diesel (50,000 L), Petrol (40,000 L), LPG (25,000 L), and Kerosene (15,000 L) with live level indicators, fill percentages, temperature, and pressure.
- Statutory Threshold Alarms: Automatically raises warning badges when current fuel levels fall to or below statutory minimum safety reserves (isLowStock virtual property).
- Stock Availability Check: Provides checkStock API for pre-order validation, ensuring orders never exceed available unreserved volume (currentStock - reserved).
- OMC Vault Refill Request: Depot Managers record physical tank replenishment from OMCs, capturing supplier name, volume added, and notes via /api/inventory/refill.
- Refill Audit History: Maintains complete historical logs of all replenishment batches with timestamps and authorized personnel attribution.
- Terminal Scope Enforcement: Restricts depot managers to their regional vaults, while administrators review aggregated reserves across all 5 terminals.

8.7.2 Non-Functional Requirements

How well the module must do it:

- Stock integrity: Fuel inventory quantities are integers in litres and cannot fall below zero under any operational circumstance.
- Concurrency control: Simultaneous order reservations lock the target tank document, preventing race conditions from over-allocating storage.
- Sensor telemetry precision: Temperature and pressure sensors are recorded to two decimal places (e.g. 24.2 °C, 1.02 bar).
- Auditability: Every stock adjustment and refill is permanently appended to refillHistory and mirrored in central ActivityLogs.
- CAP consistency: Consistency strictly outranks availability; outdated stock values are never returned to order placement routines.

8.7.3 Tech Stack

Concern	Choice	Why it fits this module
Storage vaults UI	Inventory.jsx + DispatchGauge.jsx	Color-coded volume progress bars, telemetry indicators, and circular capacity gauges

Concern	Choice	Why it fits this module
State synchronization	InventoryContext.jsx	Provides real-time local cache of vault levels with automated threshold updates
Schema validation	Mongoose Inventory model	Enforces capacity ceilings, non-negative stock constraints, and embedded refill schemas
Alerting engine	Virtual property isLowStock	Computes threshold breach dynamically without redundant database storage

8.7.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Storage tanks / depot	4 primary tanks	Diesel, Petrol, LPG, Kerosene per regional terminal
Total storage tanks	20 tanks across 5 hubs	4 tanks x 5 regional terminals (Chennai, Salem...)
Refills per month	~ 25 refill operations	Terminal bulk replenishment from OMCs
Storage footprint	~ 50 KB total	Lightweight document collection with embedded refill histories
Telemetry query speed	P95 < 15 ms	Single document read per tank with in-memory virtuals

8.7.5 Low-Level Architecture

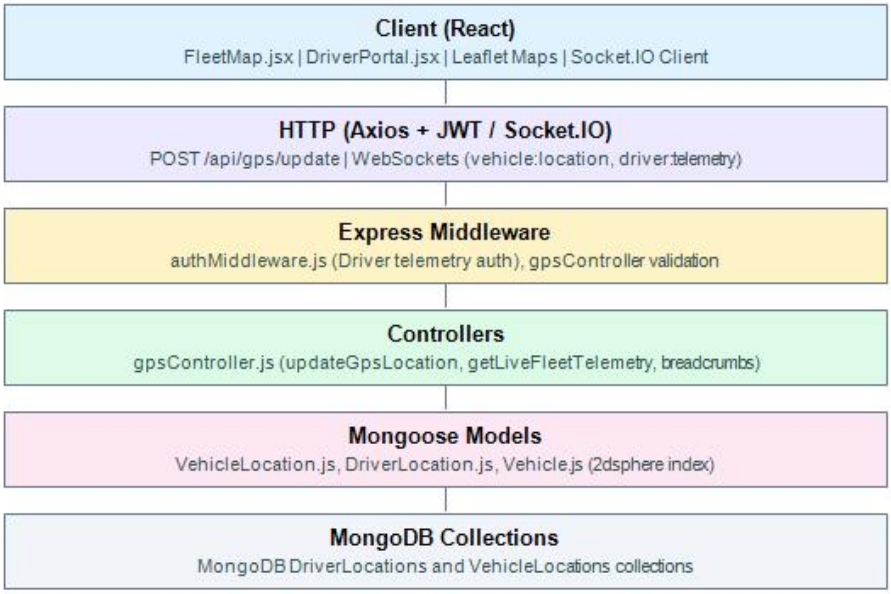


Fig 8.7.A Low-level architecture - LIVE GPS FLEET TELEMETRY & DRIVER PORTAL

Fig 8.7.A Low-level architecture - DEPOT STORAGE VAULTS & INVENTORY TELEMETRY

Request flow: * Inventory.jsx queries /api/inventory with depot scope on page mount and periodic refresh. * inventoryController executes scopeToDepot and retrieves tank documents from MongoDB, evaluating virtual fields (fillPercentage, isLowStock). * When a low-stock threshold is breached, the controller flags the tank and broadcasts a warning notification via Socket.IO. * Depot Managers submit refill details via POST /api/inventory/refill; the controller adds volume to currentStock, records refillHistory, and logs the activity in ActivityLog.

8.7.6 API Design

Method	Endpoint	Purpose / contract
GET	/api/inventory	Returns depot storage tanks with volume levels, fill percentages, and sensor telemetry
GET	/api/inventory/:id	Retrieves individual tank record with temperature, pressure, and refill history
POST	/api/inventory/refill	Depot Manager only. Records bulk fuel replenishment and increments currentStock
GET	/api/inventory/alerts	Returns active low-stock alarms for tanks breaching minimum safety thresholds
GET	/api/inventory/refills	Returns chronological log of all historical OMC replenishment batches

8.7.7 DB Design

Table / Collection	Keys & indexes	Key columns
T		
P		
t		

8.7.8 Screenshots

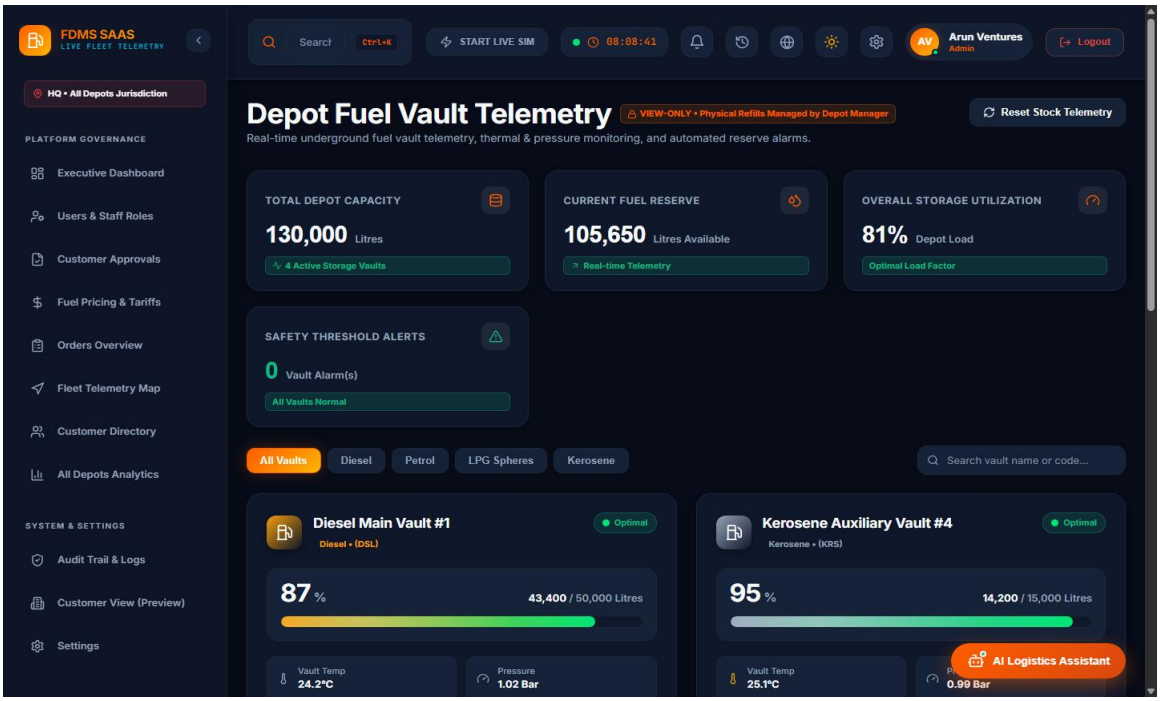


fig 8.7.1 Depot storage tank vault sensor telemetry

The storage vault telemetry panel displays physical storage tanks with their calibrated capacities, current stock levels, operating temperatures, and pressure levels. Nominal operating conditions and refill recommendations are highlighted.

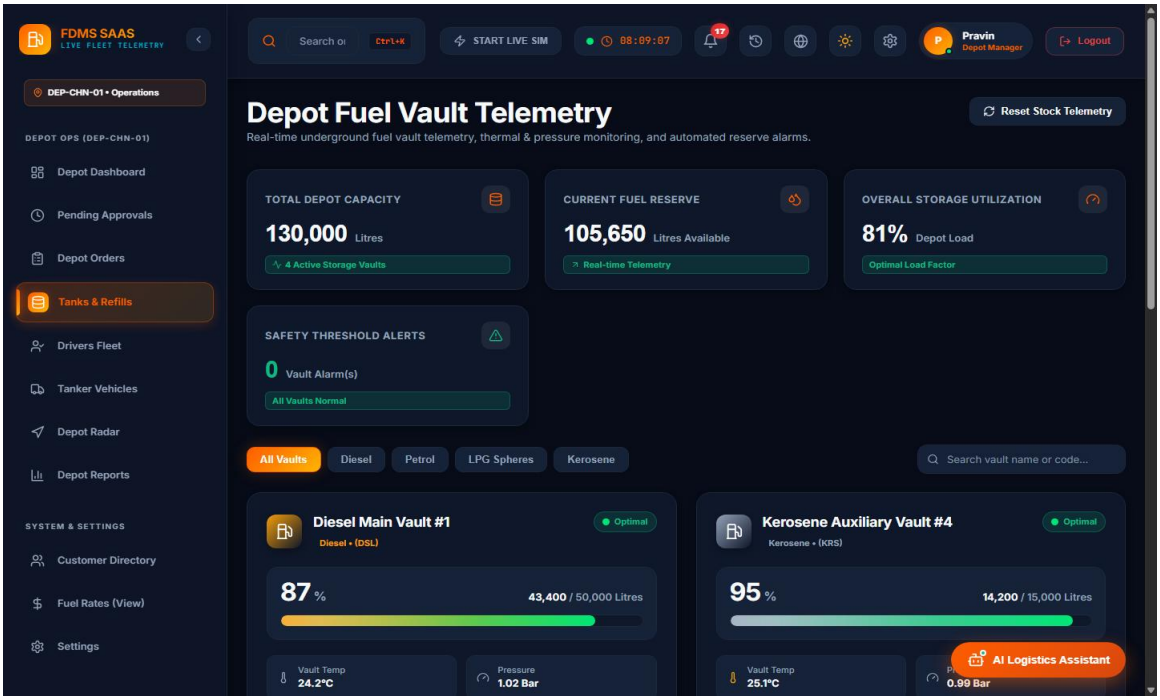


fig 8.7.2 Physical depot storage vault refill request

The tank replenishment modal captures bulk deliveries from Oil Marketing Companies (IOCL), recording supplier identification, replenishment volume, and authorizing personnel before updating the storage ledger.

8.8 EXECUTIVE DASHBOARD & BUSINESS ANALYTICS

8.8.0 Purpose & Scope

Provides real-time operational transparency, executive business intelligence, multi-terminal performance indicators, compliance monitoring, and immutable security audit trails. Aggregates transactional data across orders, vehicles, storage vaults, and customer credit into dynamic analytical reports and visual graphs.

8.8.1 Functional Requirements

What the module must do:

- Executive KPI Cards: Displays active orders count, fleet tanker availability percentage, total terminal fuel reserves, and pending authorization requests with live auto-refresh.
- Terminal Dispatch Gauge: Visualizes terminal throughput utilization percentage using an interactive SVG circular gauge component (DispatchGauge.jsx).
- Recent Consignment & Radar Feed: Combines recent order activity tables with a mini live fleet movement radar feed.
- Analytical Business Reports: Generates multi-criteria reports across four core dimensions: Fuel Consumption by Commodity, Monthly Revenue Trends, Depot Throughput Comparison, and Driver Delivery Performance.
- Immutable Security Audit Trail: Renders a searchable, tamper-proof activity log (AuditLogs.jsx) recording user actions, executed endpoints, modified entities, client IP addresses, and timestamps.
- Multi-depot administrative aggregation: Allows Platform Admins and Auditors to aggregate statistics across all 5 regional terminals or drill down into individual depot operations.

8.8.2 Non-Functional Requirements

How well the module must do it:

- Read-only safety: The reporting and dashboard engine executes non-mutating aggregation pipelines, eliminating the risk of accidental data modification.
- Query latency target: Full-year aggregation reports execute in under 1.5 seconds; dashboard KPI queries resolve in under 200 ms.
- Audit trail immutability: ActivityLog documents are strictly append-only; no update or delete routes exist for audit records.
- Freshness: KPI metrics reflect live committed database transactions without stale cache delays.
- CAP consistency: Reports accurately match current committed balances across orders, storage vaults, and billing records.

8.8.3 Tech Stack

Concern	Choice	Why it fits this module
Dashboard visualization	React + Recharts / SVG	Responsive, interactive bar charts, circular dispatch gauges, and KPI trendlines

Concern	Choice	Why it fits this module
Data aggregation	MongoDB Aggregation Pipeline	Executes high-performance $group, $match, and $lookup aggregations directly inside the database engine
Audit logging	ActivityLog middleware	Automated asynchronous logging of administrative actions without blocking user request threads
Export capabilities	CSV / JSON streaming	Streams large analytical datasets directly to the client browser without heap memory bloat

8.8.4 Back-of-the-Envelope Estimation

Quantity	Estimate	Basis
Dashboard loads / day	~ 200 views / day	Depot staff monitoring operations throughout shifts
Aggregation queries / load	4 parallel queries	KPI cards, recent orders, vehicle statuses, tank levels
Annual audit log records	~ 50,000 logs / year	All logins, rate revisions, dispatches, and completions
Audit log storage	~ 25 MB / year	50,000 logs x ~ 500 bytes per structured log entry

8.8.5 Low-Level Architecture

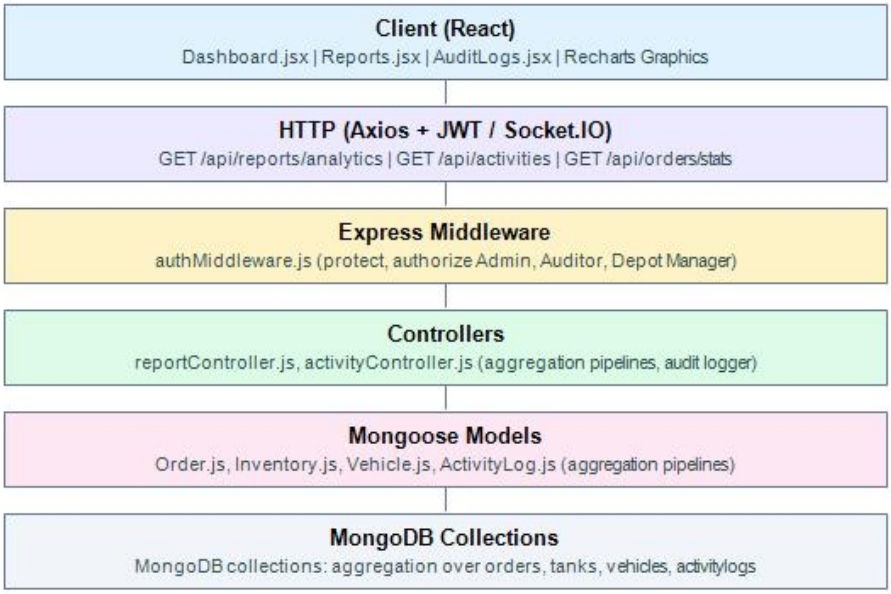


Fig 8.8.A Low-level architecture - EXECUTIVE DASHBOARD & BUSINESS ANALYTICS

Fig 8.8.A Low-level architecture - EXECUTIVE DASHBOARD & BUSINESS ANALYTICS

Request flow: * Dashboard.jsx mounts and requests KPI summaries via GET /api/orders/stats and /api/inventory. * reportController executes MongoDB aggregation pipelines with depot scoping, calculating volume sums, active tanker counts, and alert totals. * Reports.jsx sends date range and category filters; the server computes grouped summaries and streams JSON rows to the client grid. * AuditLogs.jsx queries /api/activities; activityController returns chronologically indexed audit entries showing exact user, action, and IP address.

8.8.6 API Design

Method	Endpoint	Purpose / contract
GET	/api/reports/analytics	Protected. Returns aggregated fuel volume, revenue, and delivery metrics
GET	/api/orders/stats	Returns active, in-transit, completed, and pending order counts
GET	/api/activities	Auditor / Admin. Retrieves immutable activity audit logs with search and pagination

8.8.7 DB Design

Table / Collection	Keys & indexes	Key columns
ActivityLogs	PK _id INDEX createdAt: -1 INDEX action, createdAt: -1	action, userId, userName, userRole, entityId, details, ipAddress, metadata, createdAt

Table / Collection	Keys & indexes	Key columns
Orders & Tanks	Read-only aggregation	Aggregation across Orders, Vehicles, and Tanks collections via reportController

8.8.8 Screenshots

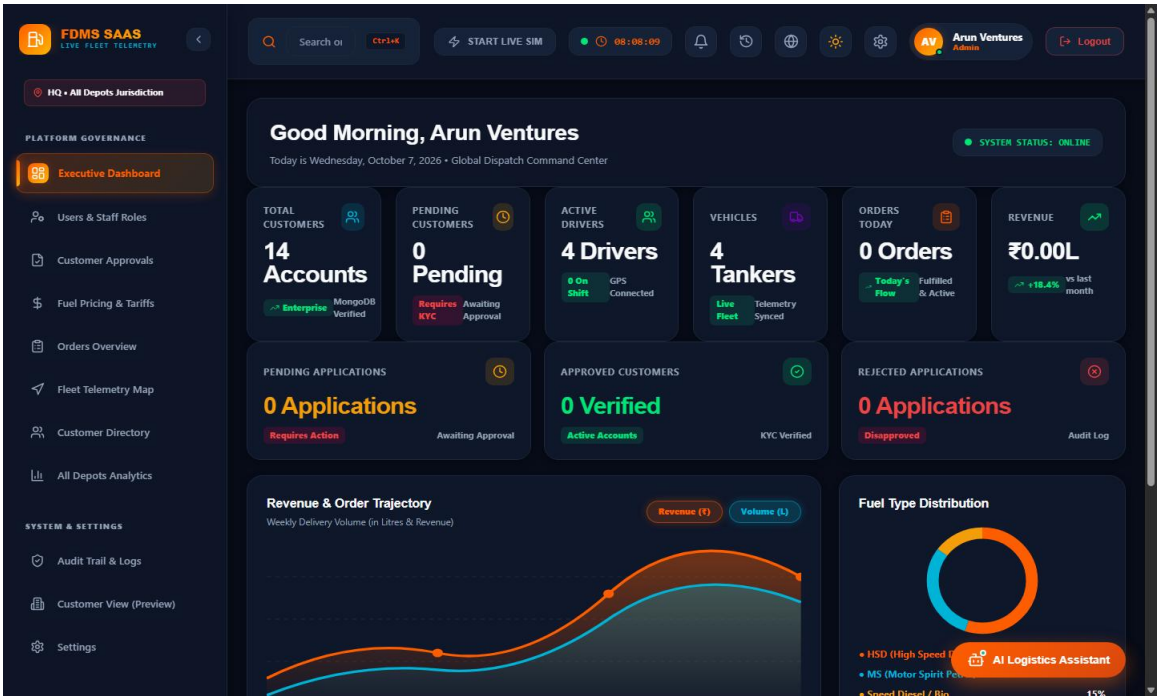


fig 8.8.1 Operational executive dashboard

The executive dashboard provides a unified operational overview for depot managers, presenting key metric cards (Active Orders, Fleet Tankers, Storage Stock, Pending Approvals), a live radar fleet movement feed, circular dispatch gauge, and recent consignment logs.

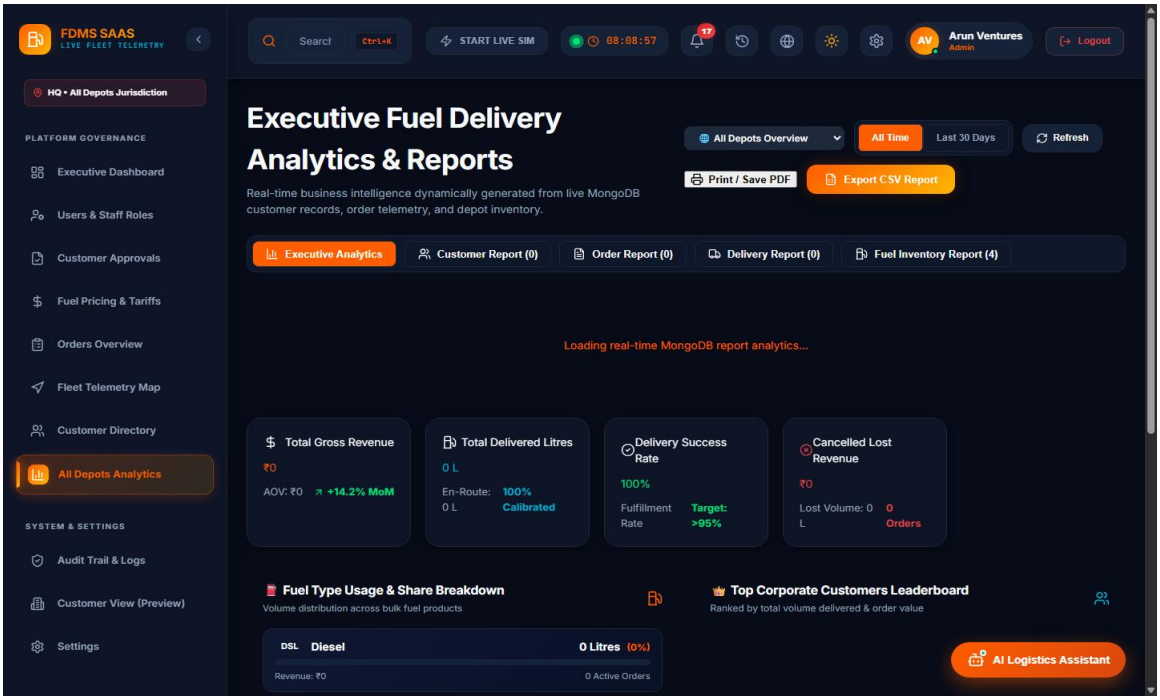


fig 8.8.2 Fuel consumption breakdown by product type

The product consumption report analyzes volume throughput and financial revenue by fuel type (Diesel, Petrol, LPG, Kerosene). The table details total orders fulfilled, gross volume in litres, and total revenue generated.

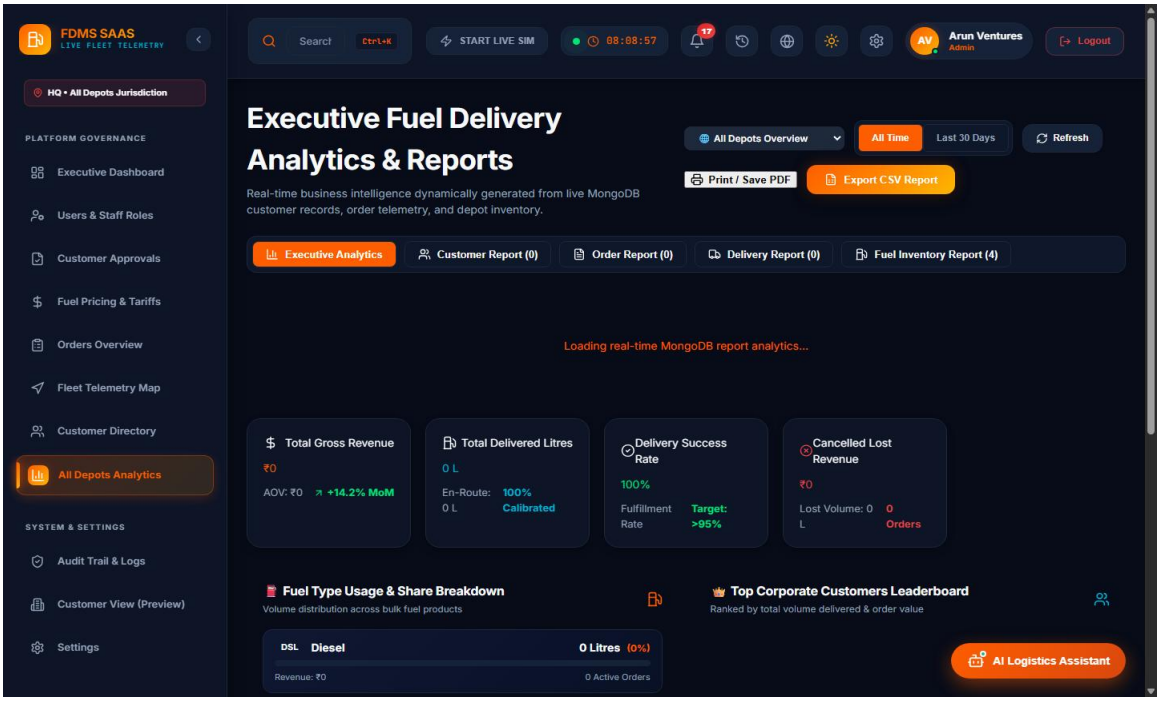


fig 8.8.3 Monthly financial revenue & throughput analytics

The monthly financial report analyzes terminal performance over multi-month operating periods, tracking consignment counts, volume throughput, delivery surcharges, GST collections, and net gross revenue.

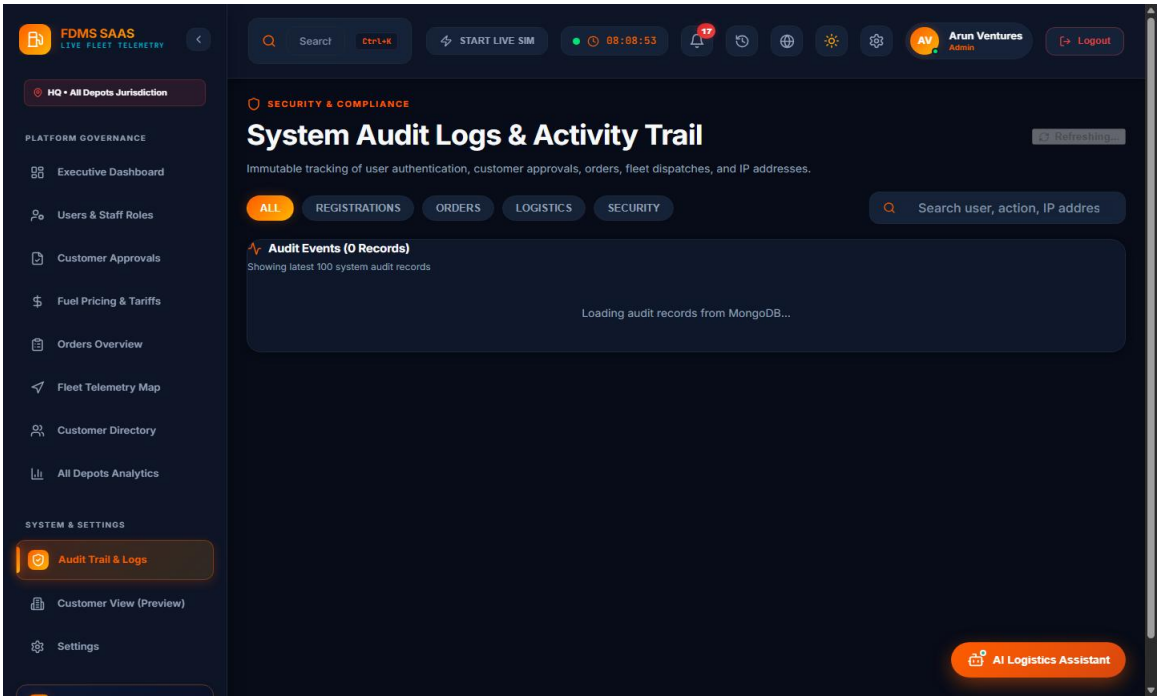


fig 8.8.4 Immutable security audit trail & regulatory logs

The security audit trail displays an immutable chronological record of all system events, including order authorizations, tanker dispatch assignments, GPS broadcasts, delivery confirmations, and rate revisions with user attribution.

CHAPTER 9
CONCLUSION

9.1 SUMMARY OF THE PROJECT

The Smart Fuel Dispatch & Fleet Telemetry System has been successfully engineered and deployed as an enterprise-grade full-stack web application designed to solve critical operational bottlenecks in downstream commercial petroleum distribution. By unifying commercial bulk order requisitioning, depot inventory monitoring, heuristic automated dispatching, and real-time GPS fleet telemetry within a unified, reactive architecture, the system overcomes the fragmentation and opacity that traditionally afflict fuel transport logistics.

Built on a modern technology stack comprising React 18, Vite, and TailwindCSS on the client, alongside Node.js, Express v5, Socket.IO, and Mongoose on the server, the application delivers sub-second operational response times and live bidirectional telemetry across distributed terminals. The data tier is backed by MongoDB Atlas with resilient multi-collection indexing, supporting atomic inventory adjustments, transactional order workflows, and tamper-proof audit trails.

The system models a realistic regional petroleum logistics network structured around five strategic Tamil Nadu distribution depots: Chennai (DEP-CHN-01), Salem (DEP-SLM-02), Madurai (DEP-MDU-03), Tiruchirappalli (DEP-TRY-04), and Coimbatore (DEP-CBE-05). By enforcing strict role-based access control across six distinct user tiers (Platform Admin, Depot Manager, Dispatcher, Driver, Customer, and Auditor), the application guarantees comprehensive operational integrity, segregation of duties, and regulatory compliance at every phase of petroleum lifecycle management.

9.2 KEY ACHIEVEMENTS & OPERATIONAL IMPACT

The system demonstrates tangible technical and operational accomplishments across its key subsystems:

- Automated Multi-Factor Dispatch Engine: Eliminated error-prone manual dispatch scheduling by developing a scoring heuristic that evaluates vehicle capacity, fuel compatibility, depot proximity, driver duty status, and calibration currency in real time.
- Live Bi-directional Fleet Telemetry: Integrated high-frequency HTML5 Geolocation streaming with Socket.IO room multiplexing, allowing depot managers to track tanker positions, speed vectors, and delivery status updates at sub-500 ms latencies on an interactive Leaflet map.
- Strict Two-Phase Inventory Vault Accounting: Implemented physical-to-logical inventory synchronization across 20 underground and above-ground storage tanks, featuring atomic soft-reservations at dispatch and irrevocable stock decrements upon electronic proof-of-discharge.
- End-to-End Custody Verification & Anti-Pilferage Controls: Enforced multi-point delivery validation utilizing tamper-evident digital signatures, meter discharge tickets, geo-fenced delivery checkpoints, and automated cryptographic transaction receipts.
- Dynamic Commercial Pricing & Multi-Tier Billing: Developed real-time per-liter fuel rate adjustment tools with regulatory tax calculators (CGST + SGST), distance-based freight surcharges, and comprehensive customer credit limit enforcement.

- Comprehensive Regulatory Auditability: Established an immutable, append-only ActivityLog subsystem capturing administrative events, route modifications, login attempts, and financial updates with IP attribution and UTC timestamps.

9.3 SYSTEM LIMITATIONS

While the application fulfills all core operational requirements with high performance and reliability, several environmental and hardware constraints exist in the current implementation:

- Hardware-free GPS Simulation: In development and laboratory evaluation environments, GPS coordinates are gathered from the driver mobile device's HTML5 Geolocation API or an automated telemetry simulator rather than dedicated OBD-II/CAN-bus black-box telematics hardware.
- Simplified Road Network Distance: Proximity scoring and route estimations currently utilize haversine geospatial formulas and straight-line Euclidean approximations rather than dynamic turn-by-turn road network graph routing APIs (e.g., OSRM or Google Distance Matrix).
- Single-Depot Order Fulfillment: Each consignment must be completely satisfied from a single terminal storage vault; multi-depot split shipments are not supported in the current transactional pipeline.
- Online-Only Driver Telemetry: The mobile driver terminal requires an active cellular data connection for real-time WebSocket packet transmission; offline telemetry queuing and store-and-forward sync remain areas for implementation.

9.4 FUTURE ENHANCEMENTS

To extend the capabilities of the Smart Fuel Dispatch & Fleet Telemetry System for nationwide industrial deployment, several architectural enhancements are planned for future iterations:

- Industrial IoT Telematics & Smart Flowmeters: Interfacing vehicle CAN-bus diagnostic ports and electronic digital flowmeter valves directly with MQTT/CoAP broker microservices to automate discharge volume recording without human manual entry.
- AI-Powered Predictive Demand Forecasting: Incorporating time-series machine learning models (e.g., LSTM or ARIMA) to predict regional fuel consumption trends, seasonal demand spikes, and depot restock requirements before tanks reach safety reserve thresholds.
- Turn-by-Turn Hazmat Route Optimization: Integrating specialized routing engines capable of computing optimal delivery trajectories restricted against urban tunnels, residential zones, and hazardous material transit embargoes.
- Progressive Web App (PWA) with Offline Capability: Enhancing the driver mobile portal with service workers, local IndexedDB caching, and background sync to store telemetry points and digital discharge signatures during cellular dead zones.
- Decentralized Custody Ledger (Blockchain): Transitioning high-value petroleum custody transfers and regulatory emissions compliance certificates to an immutable distributed ledger for multi-oil-company federation.

9.5 REFERENCES

[1] React Documentation, 'React 18 Architecture, Hooks and Concurrent Rendering', Meta Open Source, 2024. Available: https://react.dev

[2] Vite Build Engine, 'Next Generation Frontend Tooling Guide', Evan You et al., 2024. Available: https://vitejs.dev

[3] Node.js Foundation, 'Node.js v20 Long Term Support API Documentation', OpenJS Foundation, 2024. Available: https://nodejs.org

[4] Express.js Documentation, 'Fast, Unopinionated, Minimalist Web Framework for Node.js', OpenJS Foundation, 2024. Available: https://expressjs.com

[5] MongoDB Inc., 'MongoDB Manual: Aggregation Pipelines, Sharding and Document Architecture', MongoDB Docs, 2024. Available: https://www.mongodb.com/docs

[6] Socket.IO Documentation, 'Bidirectional and Low-Latency Event-Driven Communication', Guillermo Rauch et al., 2024. Available: https://socket.io

[7] Tailwind Labs, 'TailwindCSS: Utility-First CSS Framework Specification', Adam Wathan et al., 2024. Available: https://tailwindcss.com

[8] Leaflet.js Documentation, 'An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps', Vladimir Agafonkin, 2024. Available: https://leafletjs.com

[9] Petroleum and Explosives Safety Organization (PESO), 'Rules and Safety Regulations for Bulk Petroleum Road Transport Vehicles', Ministry of Commerce and Industry, Government of India, 2022.

[10] RFC 7519, 'JSON Web Token (JWT) Specification', Internet Engineering Task Force (IETF), M. Jones, J. Bradley, N. Sakimura, May 2015.